Ingegneria del sw D.P.

Design Pattern

Definizioni e caratteristiche

- I **Design Pattern** sono strutture software (ovvero micro-architetture) per un piccolo numero di classi che descrivono **soluzioni di successo per problemi ricorrenti.** Tali micro-architetture specificano le diverse classi ed oggetti coinvolti e le loro interazioni.
- Si mira a riusare un insieme di classi, ovvero la soluzione ad un certo problema ricorrente, che spesso è costituita da più di una classe.
- Durante la progettazione, prendiamo diverse scelte sulle classi, le cui conseguenze potrebbero non essere subito note, e le classi potrebbero diventare difficili da riusare o non esibire alcune proprietà (**riusabilità, estensibilità, sostituibilità).** Con sostituibilità intendiamo sostituire la classe in questione con un'altra classe compatibile senza rompere il codice che la usa. Per esempio, sostituire una LinkedList con un ArrayList.
- Esistono tanti cataloghi di Design Pattern, per vari contesti: sistemi centralizzati, concorrenti, distribuiti, real-time, etc.pensato per **sistemi orientati agli oggetti centralizzati** (un singolo processo, un singolo thread logico).
- Un Design Pattern descrive un problema di progettazione ricorrente che si incontra in specifici contesti e presenta una **soluzione collaudata generica ma specializzabile.** Documentano quindi soluzioni già applicate che si sono rivelate di successo per certi problemi e che si sono evolute nel tempo.
- **Aiutano i principianti** ad agire come se fossero esperti.
- **Supportano gli esperti** nella progettazione su grande scala.
- Evitano di re-inventare concetti e soluzioni, riducendo il costo.
- Forniscono un **vocabolario comune** e permettono una comprensione dei principi di design.

- Non si occupano solo di cosa fa il codice (**requisiti funzionali**), ma analizzano le loro **proprietà non-funzionali** (come è realizzato): ovvero, come una funzionalità è realizzata, es. affidabilità, modificabilità, sicurezza, testabilità, riuso.

Descrizione di un Design Pattern

Un Design Pattern nomina, astrae ed identifica gli aspetti chiave di un **problema di progettazione**, le classi e le istanze che vi partecipano, i loro ruoli e come collaborano, ovvero la distribuzione delle responsabilità.

La descrizione include cinque parti fondamentali:

1. **Nome:** permette di identificare il Design Pattern con una parola e di lavorare con un **alto livello di astrazione**, e indicando lo scopo del pattern.
2. **Intento:** descrive brevemente le **funzionalità** e lo **scopo**.
3. **Problema (Motivazione + Applicabilità):** descrive il problema a cui il pattern è applicato e le condizioni necessarie per applicarlo.
4. **Soluzione:** descrive gli elementi (classi) che costituiscono il design pattern, le loro responsabilità e le loro relazioni.
5. **Conseguenze:** indicano risultati, compromessi, vantaggi e svantaggi nell'uso del Design Pattern.

Nota: nella sezione Problema si parla di **forze** (come in fisica). Ovvero, le forze sono **obiettivi e vincoli,** spesso contrastanti, che si incontrano nel contesto del Design Pattern.Una **forza** è un obiettivo o vincolo che il tuo design deve soddisfare. Il problema è che spesso queste forze **si contraddicono a vicenda**, proprio come in fisica due forze possono tirare in direzioni opposte. Il pattern è la **soluzione che bilancia queste forze**.

Altre parti della descrizione di un Design Pattern possono essere:

- **Esempi di utilizzo:** illustrano dove il design pattern è stato usato.
- **Codice:** fornisce porzioni di codice che lo implementano.

Organizzazione

I design pattern sono organizzati sul catalogo (libro **GoF**) in base allo scopo. Ne esistono di diversi tipi:

- **Creazionali:** riguardano la creazione di **istanze**. Fanno parte di questa categoria design pattern come il Singleton, il Factory Method, l'Abstract Factory, Builder.
- **Strutturali:** riguardano la scelta della **struttura**. Fanno parte di questa categoria design pattern come l'Adapter, il Facade, il Composite, il Decorator, il Bridge.
- **Comportamentali:** riguardano la scelta dell'**incapsulamento di algoritmi**. Fanno parte di questa categoria design pattern come l'Iterator, il Template Method, l'Observer, lo State.

Categorie di Design Pattern

Design Pattern Creazionali

I design pattern creazionali presentano le seguenti caratteristiche:

- Permettono di **astrarre il processo di creazione oggetti**: rendono un sistema indipendente da come i suoi oggetti sono creati, composti e rappresentati.
- Sono importanti se i sistemi evolvono per dipendere più su **composizioni di oggetti** che su ereditarietà tra classi. L'ereditarietà è rigida e la gerarchia è fissa a compile time: la composizione, infatti, è più flessibile, dato che possiamo combinare oggetti diversi a runtime.
- L'enfasi va dal codificare un insieme fissato di comportamenti verso un più **piccolo insieme di comportamenti fondamentali componibili.** Invece di avere tante classi ognuna con un comportamento specifico fisso, bisogna avere **poche classi con comportamenti base che possiamo combinare fra loro.** Meno classi e più flessibilità.
- Incapsulano conoscenza sulle classi concrete che un sistema usa. **Il codice client non sa quali classi concrete vengono istanziate.** Questa conoscenza è nascosta dentro il pattern. Ad esempio, con un metodo factory il client chiama `createDocument()` senza sapere se ottiene un PdfDocument o un WordDocument.

Sposta la responsabilità di sapere quale classe concreta usare dal client al pattern.

- Nascondono come le istanze delle classi sono create e composte. Il codice client non sa come l'oggetto è costruito internamente: se è un singleton, se viene creato da zero o recuperato da un pool. Vede solo l'oggetto finito e pronto da usare.
- **Sintesi:** si occupano di come vengono creati gli oggetti. L'idea di fondo è **disaccoppiare il codice client dal processo concreto di istanziazione**. Invece di scrivere `new ConcreteClass()` sparso ovunque nel client (cosa che lega il codice a una classe specifica e rende difficile cambiarlo in futuro), si delega la creazione a un meccanismo dedicato. Questo permette di decidere a runtime quale classe concreta istanziare, di nascondere la complessità di costruzione di oggetti complessi, o di garantire vincoli come l'unicità di un'istanza. Si vuole **nascondere al client** come gli oggetti vengono creati, istanziati e composti.

Design Pattern Strutturali

- **Si occupano di come classi e oggetti vengono composti per formare strutture più grandi, e anche di come vengono organizzati per semplificare le relazioni fra loro**, mantenendole flessibili ed efficienti. Qui il problema è l'organizzazione: come far collaborare interfacce incompatibili, come comporre oggetti in gerarchie ad albero trattando foglie e composizioni in modo uniforme, come aggiunge responsabilità a un oggetto senza modificarne la classe, etc. Spesso sfruttano composizione e/o ereditarietà per ottenere nuove funzionalità combinando oggetti esistenti.
- Risponde alla domanda "il pattern serve a semplificare/riorganizzare una struttura di oggetti già esistente?".
- "Il pattern esiste per organizzare la forma con cui oggetti/classi si compongono in una struttura".
- L'obiettivo è **ottenere nuove funzionalità** combinando oggetti esistenti in modo flessibile, senza dover modificare le classi di partenza.

Design Pattern Comportamentali

- **Si occupano di algoritmi e dell'assegnazione di responsabilità tra oggetti, e di come comunicano a runtime.**
  Si occupano della comunicazione e della distribuzione di responsabilità fra oggetti. Come gli oggetti interagiscono, chi fa cosa, e come si propagano stati o richieste. Sono la categoria più ampia e variegata, perché il comportamento di un sistema può essere catturato in molti modi diversi: tramite algoritmi intercambiabili, notifiche tra osservatori, macchine a stati, catene di gestori, etc.
- Risponde alla domanda "il pattern serve a introdurre/incapsulare una logica di variabilità nel comportamento o nel flusso di controllo a runtime?".
- "Il pattern esiste per gestire/assegnare responsabilità di comportamento (algoritmi, decisioni su chi fa cosa) in modo che diventi intercambiabile o configurabile".
- Si occupa di **come gli oggetti collaborano e distribuiscono le responsabilità** fra loro. Non riguardano tanto la struttura statica del sistema, ma più che altro il flusso di controllo e la comunicazione a runtime fra gli oggetti.

Suddivisione dei Design Pattern

D.P. Creazionali: Factory Method, Abstract Factory, Prototype, Singleton.

D.P. Strutturali: Adapter, Bridge, Composite, Decorator, Facade.

D.P. Comportamentali: Chain of Responsibility, Command, Mediator, Observer, State, Template Method.

D.P. Architetturale (a parte, non è compreso nei GoF originali): MVC.

DP: Factory Method

Intento: definire una interfaccia per creare un oggetto, ma lasciare che le sottoclassi decidano quale classe istanziare. Factory Method **permette ad una classe di rimandare l'istanziazione alle sottoclassi.**

Problema:

- Un framework usa classi astratte per definire e mantenere relazioni tra oggetti. Il framework deve creare oggetti ma conosce solo classi astratte che non può istanziare. Un metodo responsabile per l'istanziazione (detto **factory**, ovvero

fabbricatore) incapsula la conoscenza su quale classe creare. Quindi il framework lavora con classi astratte perché deve essere **generico e riusabile**, ma a un certo punto dovrà creare oggetti concreti, e non può farlo da solo perché non sa quali classi concrete esistono.

- Il problema quindi è che non può istanziare Document perché è astratto, e non può istanziare un oggetto specifico perché sarebbe vincolante. Quindi ha un'idea: rimanda l'istanziazione alle sottoclassi, e così poi, tramite il **dispatch a runtime,** verrà chiamata la versione corretta di `createDocument`.
- Ci sono due **esigenze che si scontrano:**
  1. Essere **generico:** ovvero il framework vuole funzionare con qualsiasi tipo di oggetto (sottoclassi), e per questo lavora con la superclasse generica e astratta, e non si lega a nessun tipo specifico.
  2. **Creare oggetti:** vuole creare un oggetto reale da poter usare.
- La soluzione è quella di delegare la creazione dell'oggetto alla sottoclasse. Lei si occupa di chiamare il **factoryMethod** per creare l'oggetto specifico corretto.
- **Nota:** con framework in questo contesto intendiamo una **struttura di classi astratte** che definisce uno **scheletro di applicazione**, lasciando alcuni dettagli da riempire a chi usa il framework.

Soluzione:

- **Product:** è l'interfaccia comune degli oggetti creati da factoryMethod().
- **ConcreteProduct:** è un'implementazione di Product.
- **Creator:** dichiara il factoryMethod(), e quest'ultimo ritorna un oggetto di tipo Product. Creator può avere un'**implementazione di default** di factoryMethod() che ritorna un certo ConcreteProduct. In questo caso, in UML ci sarebbe una freccia da Creator verso un ConcreteProduct specifico.
- **ConcreteCreator:** implementa il factoryMethod(), o ne fa override, e sceglie quale ConcreteProduct istanziare e ritorna tale istanza. Nell'esempio di sotto, il ConcreteCreator conosce entrambi i ConcreteProduct, e può istanziare quello che gli serve a seconda della logica interna.
  Tipicamente, esiste un ConcreteCreator per un singolo ConcreteProduct, quindi si parla di corrispondenza **1-a-1** (approccio più semplice e leggibile).

Ogni ConcreteCreator esiste solo per creare un tipo specifico di ConcreteProduct, seguendo il principio **SRP**.

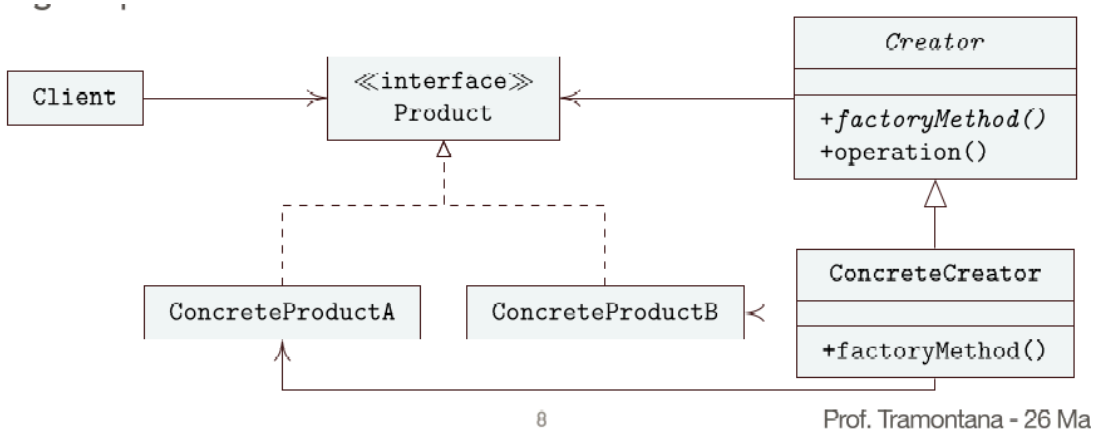


Diagramma UML delle classi

Diagramma UML di sequenza:

- `operation()`: è un metodo del Creator. Il Client chiama operation() su un riferimento che per lui è `Creator`, ma che punta a un oggetto il cui tipo dinamico è `ConcreteCreator`. Operation al suo interno ha bisogno di creare un oggetto e quindi chiama `factoryMethod()`. Operation è codice definito in Creator, ma a runtime sta eseguendo sull'istanza `cc` (ConcreteCreator). Viene chiamato il metodo `factoryMethod()` sulla stessa istanza, e per **late-binding** entra in esecuzione la versione overriden di ConcreteCreator (e non quella di Creator). La freccia con la punta nera piena rappresenta un **messaggio sincrono**, cioè una **chiamata di metodo**. Il metodo scritto sulla freccia appartiene all'oggetto verso cui punta, ed è invocato dall'oggetto da cui parte. Poi c'è anche la **freccia riflessiva**, come quella di `factoryMethod()`, dato che il concreteCreator chiama un proprio metodo su sé stesso.
- `create`: viene creato il ConcreteProductA, specificato dal ConcreteCreator. Rappresenta il **messaggio che istanzia l'oggetto**, in particolare l'**invocazione del costruttore** (sicuramente grazie al metodo chiamato poco prima, in questo caso grazie a factoryMethod).
- Successivamente, `cp` viene ritornato da factoryMethod e da operation. La freccia tratteggiata con la punta aperta rappresenta il ritorno (non è una

chiamata di metodo), e l'etichetta sulla freccia rappresenta il valore restituito. In questo caso, il valore restituito è la variabile di nome `cp` , ovvero ConcreteProductA, che viene restituita al Client.

- Successivamente, dato che il Client possiede un **riferimento a Product** che punta dinamicamente a un oggetto `ConcreteProductA`, può utilizzare i metodi di questo oggetto, e in questo caso chiama il metodo `request()`.

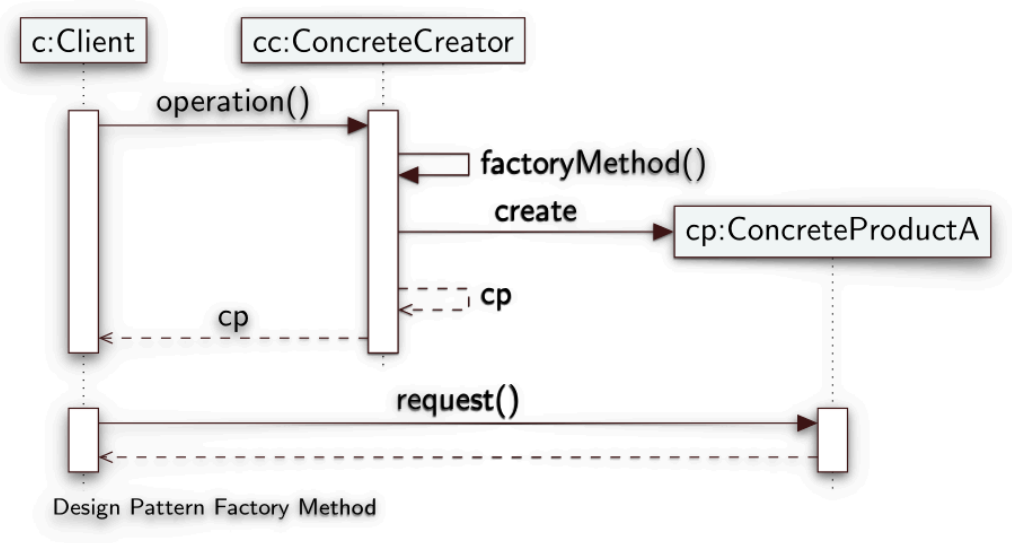


Diagramma UML di sequenza

Differenza tra UML di sequenza e UML delle classi

Il diagramma delle classi e il diagramma di sequenza non devono necessariamente combaciare freccia per freccia, perché rispondono a due domande diverse:

1. Il **diagramma delle classi** dei pattern catalogati è quasi sempre **idealizzato.** Disegna solo le dipendenze che costituiscono il cuore concettuale del pattern, e omette o semplifica il resto.
2. Il **diagramma di sequenza** mostra la **collaborazione a runtime completa**, e quindi è costretto a includere ogni messaggio reale, comprese le varie chiamate "di servizio".

Quindi, la dipendenza `Client -> Creator` presente nel diagramma di sequenza esiste davvero. Se il Client invoca `operation()` su un Creator, allora ci sono pochi dubbi da avere: il Client dipende dal Creator, punto. Un diagramma delle classi **rigoroso**

dovrebbe disegnare quella freccia. Una nota importante da considerare è che nel diagramma del Gamma questo box Client non è presente, e questa è un'aggiunta puramente didattica.

Digressioni su Product e Creator

- **Product:** è un'interfaccia perché definisce solo **cosa** un prodotto deve saper fare. I prodotti concreti **non condividono nulla**. Non c'è codice né stato comune da fattorizzare in una superclasse. Un **contratto puro** è la scelta giusta. Inoltre, un ConcreteProduct potrebbe implementare Product e altre interfacce, oppure estendere già qualche altra classe. In questo modo l'interfaccia non consuma l'unica ereditarietà disponibile. Bisogna seguire il principio di "programmare per le interfacce".
- **Creator:** è una classe astratta e non un'interfaccia perché ha bisogno di tre cose che un'interfaccia non potrebbe dargli:
  1. **Stato d'istanza**: il creatore conta quante istanze ha prodotto. Le interfacce non possono avere campi d'istanza mutabili.
  2. **Metodo concreto** `final` **con un corpo:** capita spesso, così come anche nel nostro esempio di Studente, che il Creatore abbia un metodo di creazione che non deve essere modificato. Nel nostro caso, `creaStudente()` si occupa di incrementare il contatore delle istanze e di chiamare il costruttore per lo Studente. Le interfacce non possono avere metodi final.
  3. **Combinazione** di membri concreti (`creaStudente`, `getNum`) e membri astratti (`getStudente`, `sospensi`, `ripristina`). Questo "scheletro parziale con buchi da riempire" è esattamente ciò per cui esistono le classi astratte.

Bisogna quindi capire che l'interfaccia definisce "**cosa sai fare**", mentre la classe astratta "**cosa sei in parte**". Nell'esempio precedente, Studente potrebbe essere una classe astratta e funzionerebbe, ma renderebbe la struttura più rigida senza reali vantaggi. Creator, invece, non può essere un'interfaccia, perché ha bisogno di rendere **final** il metodo `creaStudente()` e di tenere il contatore delle istanze.

Conseguenze

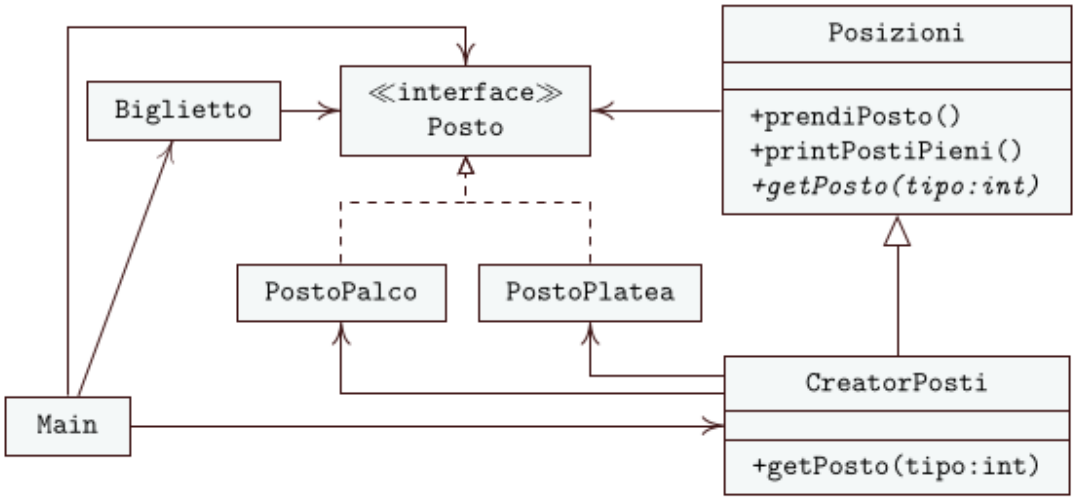
- Il codice delle classi dell'applicazione conosce solo l'interfaccia Product e può lavorare con qualsiasi ConcreteProduct. I ConcreteProduct sono **facilmente**

intercambiabili. Rappresenta il **beneficio principale.**

- Se si implementa una sottoclasse di Creator per ciascun ConcreteProduct da istanziare si ha una **proliferazione di classi.** Questo rappresenta uno svantaggio di questo design pattern, il costo che si paga per avere **disaccoppiamento, estensibilità Open/Closed** (poter aggiungere un nuovo prodotto/una nuova coppia creator-prodotto è molto semplice e si può fare senza dover modificare il codice esistente) e **classi coese senza condizionali** (ogni ConcreteCreator fa una cosa sola, senza avere un'unica factory monolitica con un grande blocco if/switch). Il costo quindi è rappresentato da un maggior numero di classi. Quindi potremmo ritrovarci con tanti ConcreteCreator quanti sono i ConcreteProduct.

Esempio Concreto di Factory Method

Abbiamo visto poco prima che nel diagramma UML "generico" e idealizzato, veniva messo in luce solo il disaccoppiamento essenziale (Client → Product), e lasciava implicita la dipendenza Client → Creator. Nell'esempio seguente, questa dipendenza non viene lasciata implicita e offre interessanti spunti di riflessione.



È interessante notare che **Main** (ovvero il Client) dipende da:

1. **Biglietto:** il Main istanzia e manipola oggetti Biglietto. L'applicazione emette un biglietto per ciascun posto assegnato. Quindi il Main deve saper creare biglietti, ed ecco perché c'è la dipendenza `Main -> Biglietto`. Biglietto, a sua volta, dipende da Posto, e per questo c'è la dipendenza `Biglietto -> Posto`. Un biglietto è per un posto, e lo tiene come Posto, tramite astrazione. Quindi Main crea un Biglietto e gli passa il Posto ottenuto.
2. **CreatorPosti:** il Main dipende da CreatorPosti (ovvero il ConcreteCreator) perché **delega a lui la creazione**: chiede un posto, e il creatore decide e costruisce il tipo concreto giusto.
3. **Posto:** il Main lavora sui posti che ottiene, e li tratta attraverso l'interfaccia Posto. I metodi del creatore restituiscono un Posto, e Main lo usa come tale (lo passa al Biglietto, legge lo Stato, etc). Questo è il disaccoppiamento lato prodotto: Main non nomina mai PostoPalco né PostoPlatea, conosce solo il contratto Posto.

In questa variante, inoltre, il Main **dipende dal ConcreteCreator,** quindi è la variante meno disaccoppiata. Se volessimo "Programmare per le interfacce" dovremmo tipizzare la variabile come Posizioni (**tipo statico**) pur tenendo a runtime un oggetto CreatorPosti (**tipo dinamico**). In questo modo otteniamo il vantaggio di star programmando per le astrazioni, ma non abbiamo disaccoppiamento pieno perché comunque abbiamo la dipendenza da CreatorPosti.

Come fa il Client a specificare il tipo di Posto? Per farlo, il Client chiama il metodo prendiPosto() di Posizioni (Creator) passando come parametro un valore. Per esempio, il valore 1 rappresenterà il PostoPalco, e tutti gli altri valori possibili il PostoPlatea. In questo modo il Client non conosce in modo diretto né PostoPalco né PostoPlatea, ma solo una convenzione che serve per "mapparli" a dei numeri interi. In questo modo, se poi dovesse cambiare l'implementazione interna di un ConcreteProduct, il Client non ne risentirebbe, dato che lui si limita a chiamare la funzione prendiPosto di Posizioni specificando il tipo di Posto attraverso un semplice numero intero.

Object Pool

Descrizione e caratteristiche di un Object Pool:

- Un **Object Pool** è un deposito di istanze già create. Un'istanza sarà estratta dal pool quando una classe client ne fa richiesta.
- Il pool può **crescere** o può avere **dimensioni fisse**. Se è a dimensioni fisse e non ci sono oggetti disponibili al momento della richiesta, allora non creo di nuovi.
- Il client restituisce al pool l'istanza usata quando non è più utile.

Il Design Pattern **Factory Method** può implementare un Object Pool:

1. I client richiedono istanze, come visto per il Factory Method. Il punto interessante è proprio questo: in ogni implementazione concreta del Design Pattern Factory Method, il **Client non crea mai nessun prodotto**, ma chiede semplicemente al ConcreteCreator di fornirgliene uno, ma il Client non sa se questo verrà creato oppure no. Nel caso dell'object pool, solitamente non viene creato, ma viene fornito al client un oggetto già esistente nel pool: **dipende dallo stato del pool** in quel momento. Significa che l'interfaccia per ottenere oggetti non cambia, e lato client non vi è alcun tipo di modifica. Il pool **riusa quando può e crea quando deve.**
2. I client dovranno indicare quando l'istanza non è più in uso, quindi riusabile. Questa, lato client, è un'aggiunta interessante che si concretizza in una **nuova responsabilità** per il client, che dovrà occuparsi di fare un passo esplicito di **rilascio**.
3. Lo stato dell'istanza da riusare potrebbe dover essere riscritto. Questo significa che prima di consegnarla a un nuovo client, il pool deve **resettarne lo stato**, altrimenti l'utente vedrebbe dati vecchi. Un oggetto se si trova nel pool allora è libero, se è usato allora non si trova nel pool. Sono queste le uniche due opzioni possibili: **o è libero e quindi è nel pool o è in uso e quindi non è nel pool.**
4. L'object pool dovrebbe essere unico: potrei usare un **Singleton.**

Dependency Injection

- Il Design Pattern Factory Method può essere usato per inserire le dipendenze (**dependency injection**) necessarie alle istanze di ConcreteProduct.

- Tramite la Dependency Injection **un oggetto (client) riceve altri oggetti da cui dipende**, questi altri oggetti sono detti **dipendenze** (del client in questo caso).
- La tecnica di Dependency Injection permette di **separare la costruzione delle istanze dal loro uso**, e in questo modo **il client non crea l'istanza** di cui ha bisogno.
- Le dipendenze vengono **iniettate al client** per mezzo di parametri nel suo costruttore. Questo permette di **evitare complicazioni derivanti da metodi setter e da controlli** per verificare che le dipendenze non siano null, di conseguenza il codice è più semplice.
- L'oggetto che fa Dependency Injection si occupa di **connettere** (fa **wiring** di) varie istanze. In un unico posto vediamo le connessioni fra gli oggetti.

Quindi, con la dependency injection l'oggetto **non crea le sue dipendenze**, ma **le riceve da fuori**. C'è una **Constructor Injection:** la dipendenza arriva al momento della creazione. Può essere `final`, è garantita **non null** e l'oggetto è valido fin dal primo istante. Con la **Setter Injection**, invece, l'**oggetto nasce incomleto** (dipendenza null), finché qualcuno non chiama il setter. Ci sono diversi problemi in quest'ultimo approccio, come dimenticarsi di chiamare il setter e che l'oggetto è mutabile e non final.

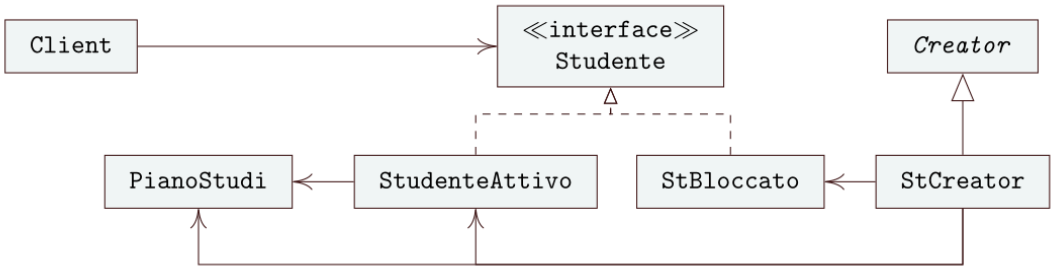
Inoltre, dato che nessuno crea le proprie dipendenze, qualcuno dovrà pur crearle e collegarle. A questo ci pensa l'**injector** (anche noto come **composition root**).

Legame col Factory Method: dato che il factory method è l'unico punto dove il ConcreteProduct viene istanziato, è il posto naturale per iniettargli quelle dipendenze. Il factory in questo modo fa un doppio lavoro: decide quale classe concreta creare e la consegna già con il **wiring** fatto con le sue dipendenze. Il client chiama `factoryMethod()` e riceve un prodotto pronto all'uso, senza conoscere né la classe concreta né le sue dipendenze. In questo caso il client che riceve le dipendenze è `ConcreteProduct`, e l'injector è il factory method stesso.
Quindi, con la Dependency Injection, il client riceve il Creator già pronto (dato che si trova nel costruttore). Infatti, chi si occuperà di creare il Client dovrà passargli nel costruttore l'istanza del ConcreteCreator scelto. In questo modo le dipendenze (ovvero il Creator, che ha come tipo dinamico un determinato ConcreteCreator) vengono iniettate all'interno di Client. Ricordiamo infatti che in questo caso il Client

conosce solamente il tipo statico Creator, ma poi dinamicamente questo si rivela essere un ConcreteCreator.

Esempio di Dependency Injection:

In questo abbiamo abbiamo che PianoStudi è una dipendenza di StudenteAttivo. Quindi nel costruttore di StudenteAttivo è presente PianoStudi, ma questa dipendenza verrà iniettata da chi si occupa di creare StudenteAttivo, ovvero StCreator, ovvero il factoryMethod del ConcreteCreator. StCreator costruirà StudenteAttivo e gli inietterà all'intero PianoStudi.



Dependency Injection

Dependency Injection: Sintesi

- **Idea centrale:** un oggetto non crea le proprie dipendenze, ma le riceve dall'esterno. Questo separa la costruzione dall'uso.
- Quando arriva la dipendenza? **Al momento della creazione**.
- L'oggetto è garantito **non-null**.
- L'oggetto è quindi valido fin da subito.
- Il **rischio** di non usare la dependency injection è quello di **dimenticare di usare il setter** per configurare l'oggetto.
- Dato che nessuno crea le proprie dipendenze, qualcuno deve farlo. Questo ruolo spetta all'**injector**. Istanzia tutti gli oggetti necessari, e li **connetti (wiring)** in un unico posto. Il `factoryMethod()`, dentro il ConcreteCreator, è il posto naturale per fare injection perché è l'unico punto dove il ConcreteProduct viene istanziato.

- Il Client chiama `factoryMethod()` e riceve un oggetto pronto all'uso, **senza conoscere né la classe concreta né le sue dipendenze**.

DP: Abstract Factory

Intento: fornire un'interfaccia per **creare famiglie di oggetti** che hanno qualche relazione senza specificare le loro classi concrete.

Problema:

- Il sistema complessivo dovrebbe essere **indipendente dalle classi usate,** così da essere configurabile con una di varie famiglie di classi. Le classi di una famiglia dovrebbero essere usate in modo consistente.
- Per esempio, lo strato di interfaccia utente permette vari tipi di look-and-feel (ovvero le varie **identità visive e interattive complessive**), così differenti comportamenti per accessori (widget, riquadri interattivi) dell'interfaccia utente sono possibili.
- Il sistema deve poter essere configurato con **una di più famiglie** possibili, e una volta scelta una famiglia, tutti i suoi pezzi devono provenire da quella famiglia. Quindi servono due cose insieme:
  1. Poter cambiare l'intera famiglia in un colpo solo (cambio **look-and-feel**).
  2. Garantire che non si mescolino pezzi di famiglie diverse (**coerenza**).

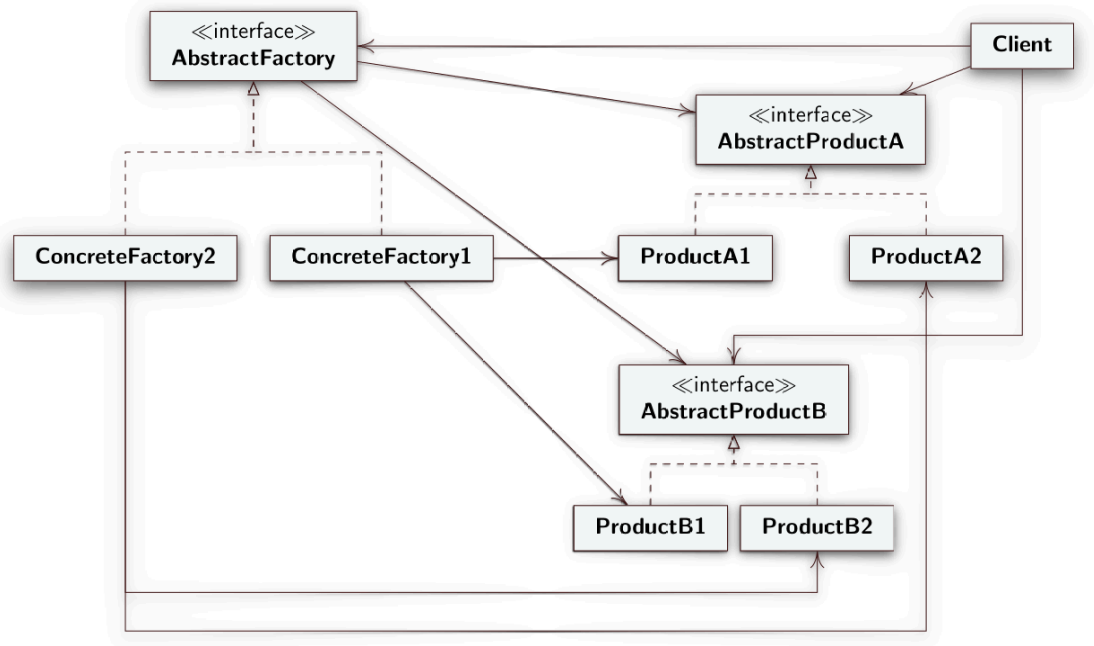
Relazione col Factory Method:

- Il Factory Method crea un prodotto, mentre l'Abstract Factory crea un'intera **famiglia di prodotti correlati**, che devono essere usati insieme in modo coerente, senza che il client conosca le classi concrete. Quindi l'Abstract Factory crea **più tipi** di prodotto correlati (e quindi un metodo per ciascun tipo).
- Mentre il Factory Method mi disaccoppia dalla classe concreta di un prodotto, l'Abstract Factory mi disaccoppia da un'intera famiglia di prodotti e **in più garantisce che siano usati in modo coerente**.

Soluzione:

- **AbstractFactory:** interfaccia astratta per creare famiglie di oggetti. Dichiara un metodo di fabbricazione per ciascun tipo di prodotto della famiglia.

- **ConcreteFactory:** classi che implementano operazioni per creare ciascuna famiglia di oggetti. Implementano l'interfaccia AbstractFactory, ciascuna producendo esclusivamente i prodotti della propria famiglia. Il client riceve un riferimento polimorfico a una ConcreteFactory e ne invoca i metodi di creazione senza mai sapere quale famiglia stia usando.
- **Interfacce AbstractProduct:** è l'interfaccia per una famiglia di oggetti. È una per ogni tipo di prodotto, e nascondono le implementazioni concrete. Quindi, per ogni tipo di Prodotto c'è un AbstractProduct.
- **Product:** definisce un oggetto, creato da ConcreteFactory, che implementa l'interfaccia AbstractProduct.
- **Client:** usa solo interfacce dichiarate da AbstractFactory e AbstractProduct.
- In questo modo è come se visualizzassimo i prodotti in una griglia bidimensionale, dove le righe sono i **tipi** di prodotto, e le colonne sono le **famiglie**.



Conseguenze:

- Permette di usare classi in modo consistente (per famiglie).
- Le famiglie di classi sono facilmente intercambiabili.

- Non è immediato supportare nuove classi AbstractProduct, poiché bisogna aggiungere un metodo su AbstractFactory e su ogni ConcreteFactory.

Spiegazione complessiva:

- Il problema che risolve è tipico del design di sistemi software ad oggetti: un client che si occupa di istanziare correttamente le classi con cui lavora. Il problema sorge quando il client deve poter scegliere fra più famiglie di oggetti. L'accoppiamento fra il client e una singola famiglia di prodotti diventa un **debito tecnico architetturale** significativo. A questo punto, **non può legarsi a ogni famiglia di oggetti**, sarebbe impensabile. Bisogna disaccoppiare il client e le famiglie di oggetti, e rendere facile e veloce per il client lo switch da una famiglia di oggetti all'altra. È questo il problema che questo Design Pattern risolve.
- Una **famiglia di oggetti** è un insieme di **oggetti distinti** per tipo ma correlati tra loro per appartenenza a una variante comune del sistema. I prodotti di famiglie diverse non devono essere mescolati, pena l'**incoerenza comportamentale o strutturale** del sistema.
- Un esempio tipico per l'applicazione di questo Design Pattern: sviluppare **toolkit grafico multi-piattaforma**. Il sistema deve produrre widget di interfaccia (Button, Checkbox, Scrollbar) in due varianti: Windows e macOS. Ogni variante forma una famiglia, infatti avremo i bottoni, le checkbox e le scrollbar in stile windows e gli stessi oggetti in stile macOS. Il client non può permettersi di scrivere `new WindowsButton()`, in quanto presente tre difetti gravi:
    1. **Accoppiamento alle classi concrete:** il client dipende direttamente dalla classe implementativa. Qualsiasi sostituzione richiede una modifica al codice client, violando il principio **Open/Closed Principle.** È uno dei principi **SOLID** dell'ingegneria del sw. Un'entità software, come una classe, un modulo o una funzione, deve essere **aperta all'estensione** (ovvero è possibile aggiungere nuovi comportamenti) e **chiusa alla modifica** (il codice sorgente esistente non deve essere alterato per accomodare tali estensioni). Ovviamente la chiusura rispetto alla modifica si riferisce alle classi astratte e alle interfacce, mentre le classi concrete possono naturalmente essere modificate per correggere bug o migliorare l'implementazione interna, purché rispettino il contratto.

2. **Scarsa estendibilità**: aggiungere una terza famiglia come Linux richiede di rintracciare e modificare ogni punto del codice in cui viene effettuata un'istanziazione concreta.
3. **Assenza di garanzie e coerenza:** nulla impedisce al client di creare un WindowsButton insieme a un MacScrollBar, ottenendo una combinazione incoerente che nessuna logica a runtime può facilmente rilevare.

Il problema quindi si generalizza così: come fare in modo che un sistema possa lavorare con famiglie intercambiabili di oggetti correlati, rimanendo completamente ignaro delle classi concrete che li implementano? La soluzione è quella proposta dal pattern.

Spiegazione del diagramma UML delle classi

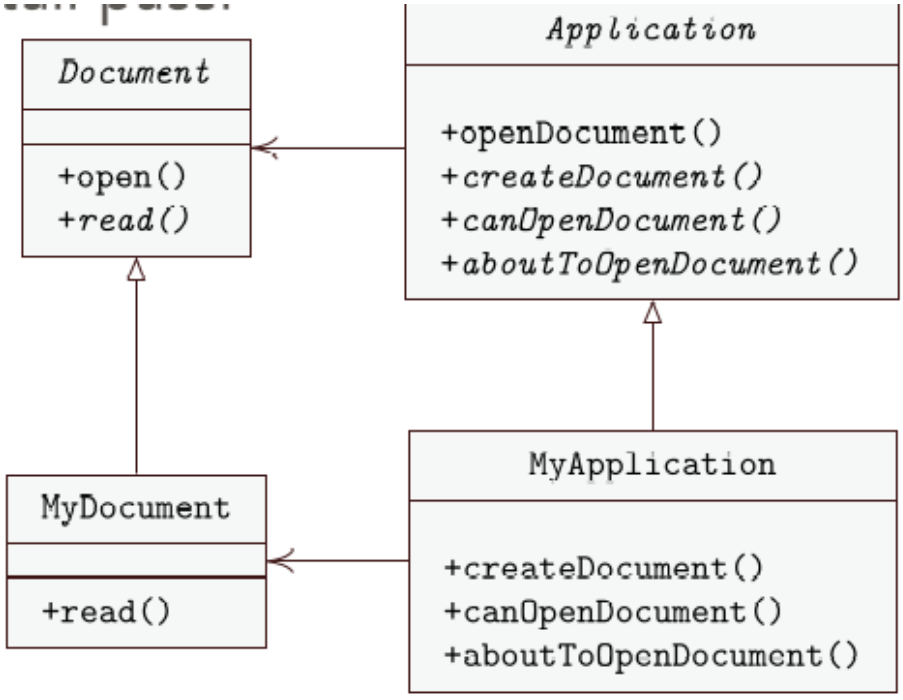
- È importante notare che in AbstractProduct la lettera indica il tipo di oggetto (es. Button), mentre il numero che poi viene usato nei Product serve per distinguere le famiglie.
- ConcreteFactory non dipende da AbstractProduct nel diagramma UML in quanto già l'AbstractFactory dipende da questi, e **la dipendenza è transitiva**. Quindi nei suoi metodi è già vincolata per contratto a restituire qualcosa che sia compatibile con AbstractProductA e AbstractProductB. Aggiungere quindi una freccia diretta da ConcreteFactory verso AbstractProductA e AbstractProductB sarebbe un'informazione ridondante, e quindi rumore visivo.
- Il Client, invece, ha comunque una freccia verso AbstractProductA e AbstractProductB, perché non si parla di dipendenza transitiva ma di **dipendenza diretta**. Il client, infatti, chiama dei metodi sopra AbstractProductA e AbstractProductB, quindi ha bisogno di conoscerli. Non basta quindi la dipendenza transitiva che deriva da AbstractFactory, ma ne serve una diretta.
- Il client, in una situazione ideale, **non conosce ConcreteFactory**, in quanto sono dipendenze che gli vengono iniettate dall'esterno (tramite la dependency injection). Infatti, solitamente dovrebbe avere un tipo AbstractFactory nel costruttore, che poi a runtime punta a un oggetto ConcreteFactory specifico.

DP: Template Method

Intento: definisce lo scheletro di un algoritmo in un'operazione e rinvia la definizione di alcuni passi nelle sottoclassi. Il Template Method lascia che le sottoclassi ridefiniscano alcuni passi di un algoritmo senza cambiare la struttura dell'algoritmo.

Problema (Motivazioni):

- Alcune volte si ha bisogno di classi che definiscono i passi necessari per un algoritmo, ma non i dettagli di tali passi.
- Per esempio, supponiamo di avere una classe Application che definisce l'algoritmo per aprire e leggere un documento in `openDocument()`, ovvero il **template method.** Tale algoritmo è definito tramite operazioni astratte che le sottoclassi implementano.
- **Template method:** metodo della superclasse che contiene lo **scheletro fisso** dell'algoritmo.

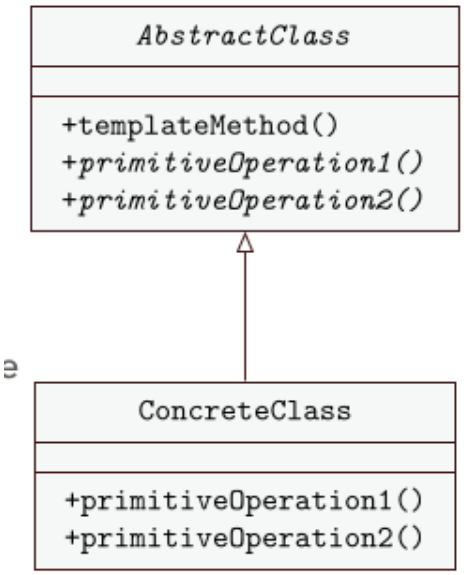


Soluzione:

- **AbstractClass** definisce operazioni primitive (come metodi astratti) che devono essere implementate dalle sottoclassi per definire i passi

dell’algoritmo. Implementa un template method che definisce lo scheletro dell’algoritmo, che chiama le operazioni astratte.

- **ConcreteClass** implementa le operazioni primitive per svolgere i passi specifici.



Conseguenze:

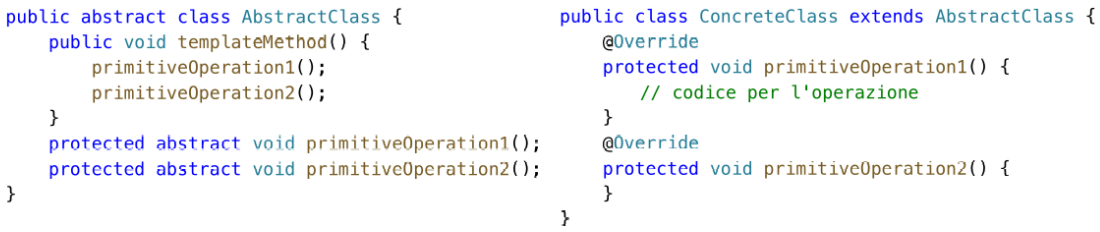
- I metodi template sono una tecnica fondamentale per riusare il codice. Sono particolarmente importanti nelle classi di libreria poiché fanno sì che si possa esprimere il comportameno comune nelle classi di libreria.
- I metodi template portano ad avere una **struttura di controllo invertita**, che **permette a una superclasse di chiamare le operazioni di una sottoclasse**, e non viceversa. **Principio Hollywood:** “non chiamarci, noi ti chiameremo”. Questo perché sarà la superclasse a chiamare la sottoclasse, e non viceversa.
- I metodi template chiamano:
  1. **Operazioni concrete su ConcreteClass:** chiama metodi che hanno già un'implementazione nella superclasse ma chiama la versione overridata della sottoclasse. **Classi client:** chiama metodi accessori che possono aiutare l'algoritmo, e sono metodi di un’altra classe.
  2. **Operazioni concrete su AbstractClass:** ovvero metodi che hanno già un'implementazione dentro AbstractClass ma che non vengono overridati,

magari ausiliari per l'algoritmo.

3. **Operazioni primitive:** proprio le operazioni che ci interessano di più, in quanto non ne viene fornita un'implementazione e spetta alla sottoclasse scriverla. **Metodi Factory:** rientra nelle operazioni primitive di cui abbiamo appena parlato, solo che si occupa di creare oggetti. È primitivo perché il suo scopo è creare un oggetto, e il tipo di oggetto dipende dalla sottoclasse specifica, quindi non può saperlo già nella superclasse. Ma niente impedisce che sia un hook se esiste un tipo di default ragionevole, ma solitamente è primitivo. Inoltre, dato che il metodo factory ha un tipo di ritorno, si potrebbe pensare di implementarlo e creare quell'oggetto, ma questa osservazione ha due problemi: il primo è che non è detto che quell'oggetto sia istanziabile, magari è un'interfaccia o una classe astratta. Il secondo problema è che non magari non esiste un oggetto "generico" che vada bene per tutti i suoi derivati, e quindi non si saprebbe come crearlo.
4. **Operazioni di aggancio (hook):** sono metodi con un corpo vuoto nella superclasse, il template method li chiama, ma di default non succede niente. La sottoclasse può sovrascriverli se vuole aggiungere un comportamento in quel punto dell'algoritmo, ma non è obbligata.
   La differenza tra operazioni primitve e di aggancio è la seguente: le operazioni primitive sono **indispensabili** per l'algoritmo, quelle di aggancio sono **opzionali** (permettono di personalizzare l'algoritmo in un determinato punto se se ne ha bisogno).

- Le operazioni primitive **devono** essere ridefinite, le operazioni di aggancio **possono** essere ridefinite, nelle sottoclassi.

Nota di dettaglio: quando diciamo qualcosa come "operazioni concrete su ConcreteClass" intendiamo operazioni concrete appartenenti a/definite in ConcreteClass.



```java
public abstract class AbstractClass {
    public void templateMethod() {
        primitiveOperation1();
        primitiveOperation2();
    }
    protected abstract void primitiveOperation1();
    protected abstract void primitiveOperation2();
}
```

```java
public class ConcreteClass extends AbstractClass {
    @Override
    protected void primitiveOperation1() {
        // codice per l'operazione
    }
    @Override
    protected void primitiveOperation2() {
    }
}
```

Codice del D.P.

Classi e Oggetti

Le classi formano il **vocabolario del dominio di un problema**. Le classi sono statiche nel senso che per loro l'esistenza, la semantica e le relazioni sono fissate prima dell'esecuzione del programma. La classe per ciascun oggetto è statica, ovvero una volta che l'oggetto è creato la classe è fissata. Gli oggetti sono creati e distrutti durante l'esecuzione di un'applicazione.

D.P. Singleton

Intento: assicurare che una classe abbia una sola istanza e fornire un punto di accesso globale all'istanza.

Motivazioni:

- Alcune classi dovrebbero avere esattamente una istanza in tutta l'applicazione. Un po' di **esempi**: uno spooler di stampa (ovvero il sistema di memorizzazione della stampante), un file system (un solo gestore che conosce lo stato del disco), un window manager (un solo gestore che sa quali finestre esistono e dove), una lista clienti (un solo oggetto che rappresenta lo stato corrente della lista), etc.
- Una **variabile globale** rende un oggetto accessibile, ma non proibisce di avere più oggetti per una classe, quindi non va bene come soluzione.
- La classe stessa dovrebbe essere responsabile di tener traccia del suo unico punto di accesso.

Soluzioni:

- La classe che deve essere un **Singleton** dovrà implementare un'operazione **getInstance() sulla classe** (quindi in Java è un **metodo static**), che ritorna l'unica istanza creata.
- La classe Singleton è **responsabile per la creazione dell'istanza**. Il Singleton, quindi, si "auto-protegge", infatti è proprio lui stesso l'istanza che ci interessa.
- Il **costruttore** della classe Singleton è **privato**, così da non permettere la creazione tramite new ad altre classi.

Chi crea il Singleton? Ci pensa proprio getInstance(). Infatti, la prima volta che viene chiamato controlla se l'istanza, ovvero **uniqueInstance**, è NULL. Se lo è, chiama il costruttore (è l'unico a poter chiamare il costruttore, visto che è privato), se invece non è NULL, allora ritorna l'istanza che già esiste, senza crearne una nuova.

Due versioni del Singleton:

1. **Eager:** l'inizializzazione dell'oggetto viene fatta non appena la classe viene caricata, e viene fatta fuori dai metodi. Si parla di **inizializzazione anticipata**. È thread-safe di default, a differenza della versione lazy.
2. **Lazy:** l'inizializzazione avviene solo quando viene chiamato il metodo getInstance(). Permette di risparmiare risorse se il Singleton non viene mai usato. È la versione di default del Singleton. Tuttavia, non è thread-safe di default, in quanto due thread potrebbero fare il controllo `if uniqueInstance == NULL` nello stesso momento, e creare quindi due istanze del Singleton.

Dubbio: stiamo chiamando un metodo su un oggetto che non esiste ancora? No, perché il metodo che stiamo chiamando, ovvero getInstance(), è statico, e infatti lo stiamo **chiamando sulla classe**, e non su un'istanza. Ricordiamo infatti che **i metodi statici sono metodi di classe**.

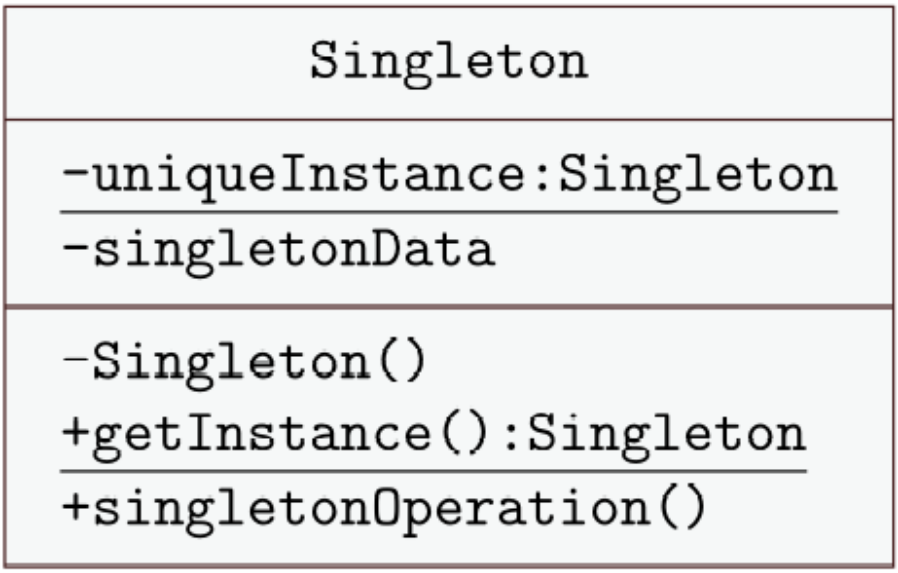


Diagramma UML del Singleton

Considerazioni e conseguenze:

- Grazie al Design Pattern Singleton, le classi non possono avere più di un'istanza a runtime.
- Nel caso dell'esempio delle slide, dove la classe che fa il ruolo del Singleton è Fib (per i numeri di Fibonacci), diremo che la classe Fib **implementa (o svolge)** il ruolo di Singleton.
- Se dopo aver realizzato una classe che svolge il ruolo di Singleton volessimo avere più istanze di quella classe anziché una sola, il codice delle classi chiamanti rimarrebbe invariato, e la sola classe da modificare è la classe che implementa il Singleton. La variante che permette un numero finito di istanze si chiama **Multiton**.
- **Principio delle Conseguenze Locali:** un cambiamento in qualche punto del codice non dovrebbe causare problemi in altri punti, questo è il principio che viene seguito anche dal Singleton, come abbiamo appena visto. I client del Singleton, infatti, chiamano getInstance con variabili diverse, ma puntano tutte allo stesso oggetto. Quando il Singleton viene modificato, queste variabili punteranno a oggetti diversi, in base al numero di istanze. Quindi i client non si accorgeranno nemmeno della modifica, in quanto la modifica sarà in termini di

comportamento, e non di codice. L'unico codice modificato sarebbe quello della classe Singleton.

- **Multiton:** supponiamo di avere la classe Pagamenti come Multiton. Potremmo associare una istanza a ciascuna cartella, e la decisione sull'istanziazione dipende quindi dall'aver già creato (o meno) un'istanza per una data cartella.
- La classe che è un Singleton ha **pieno controllo** di come e quando i client accedono al valore della sola istanza. Inoltre, evita che esistano variabili globali che tengono la sola istanza condivisa.
- Permette di controllare il numero di istanze create in un programma, facilmente e in un solo punto.
- La soluzione è più flessibile rispetto a quella di **usare static per tutte le operazioni e le variabili**, poiché si può facilmente cambiare il numero di istanze consentite. Tuttavia, la soluzione con tutto static funzionerebbe comunque. Solitamente, con questo tipo di soluzione, il costruttore si mette privato per impedire che qualcuno possa chiamarlo. In questo caso, però, nemmeno getInstance potrà chiamarlo, in quanto non esisteranno mai delle istanze della classe, ma lavoreremo solo sulla classe in modo completamente statico. Quindi **nessun costruttore chiamato**.
- L'unico frammento di codice da modificare quando si vuole variare il numero di istanze create è quello della classe che è Singleton, mentre usando static si dovrebbero modificare tutte le invocazioni.

D.P. Prototype

Idea: quando creiamo degli oggetti con `new NomeClasse()`, lo stiamo creando completamente da 0. A volte, invece di crearlo "vuoto", potrebbe esserci utile copiarne uno che esiste già. Questo oggetto che esiste già è noto come **prototipo**.

Intento: specificare i tipi di oggetti da creare usando una istanza di un prototipo, e creare nuovi oggetti copiando questo prototipo.

Motivazioni:

- **Stato iniziale complesso**: durante la creazione di un nuovo oggetto, può essere necessario avere uno stato iniziale che è simile a quello di un altro

oggetto esistente, e che quest'ultimo ha ottenuto per mezzo di operazioni eseguite su esso. Per esempio, immaginiamo di avere un oggetto `Documento` che abbiamo configurato dopo 10 operazioni (impostato il font, margini, lingua, stile, etc). Vogliamo crearne un altro uguale come punto di partenza. Ovviamente non ci conviene rifare le stesse operazioni una seconda volta, ma ha più senso clonare l'oggetto esistente.

- **Accoppiamento stretto con la classe:** quando si crea un oggetto bisogna indicare una specifica classe, e così la classe che chiede l'istanza è strettamente accoppiata con la classe da istanziare.
  Cos'è l'accoppiamento stretto? Scrivendo la seguente:

  ```
  Forma f = new Cerchio();
  ```

  Il codice **conosce esplicitamente** `Cerchio`. Se poi vogliamo che f sia un `Quadrato`, dovremo modificare quel codice. Questo è l'accoppiamento stretto. Con il Prototype, invece, il nome dell'oggetto che vogliamo creare non lo specifichiamo con il nome della classe (scrivendo `new NomeClasse()`), ma lo specifichiamo dando un'istanza già pronta da clonare. **Il tipo emerge dall'istanza, non dalla classe scritta nel codice.** Scriviamo, infatti, una cosa di questo tipo:

  ```
  Forma f = prototipo.clone();
  ```

  In questo modo, il codice che crea l'oggetto **non sa** qual è la classe concreta. Sa solo che può clonarlo. Rompe il **coupling**.

- **Difficolta ad aggiungere nuovi tipi:** lo **stretto accoppiamento** fra classi da istanziare e chi istanzia rende difficile l'aggiunta di nuovi tipi. Nel Factory Method, per esempio, bisogna creare una gerarchia di Creator per aggiungere nuovi tipi. Se avessimo, per esempio, `CerchioCreator` e `QuadratoCreator` e volessimo aggiungere `Triangolo`, dovremmo aggiungere una nuova classe `TriangoloCreator`. Quindi la gerarchia dei creator **cresce** insieme alla gerarchia dei prodotti. Col Prototype non serve nessun Creator. Per supportare Triangolo, registriamo semplicemente un'istanza prototipo di Triangolo. **Non aggiungiamo classi, aggiungiamo oggetti**.

Soluzione:

- **Prototype:** definisce l'interfaccia per le operazioni comuni e dichiara un metodo per clonare sé stessa.
- **ConcretePrototype:** classe che implementa l'interfaccia Prototype e l'operazione di clonazione.
- **Client:** classe che crea nuovi oggetti tramite clonazione dell'istanza prototipo, quindi senza indicare la classe esplicitamente. Client **conosce solo l'interfaccia Prototype**.

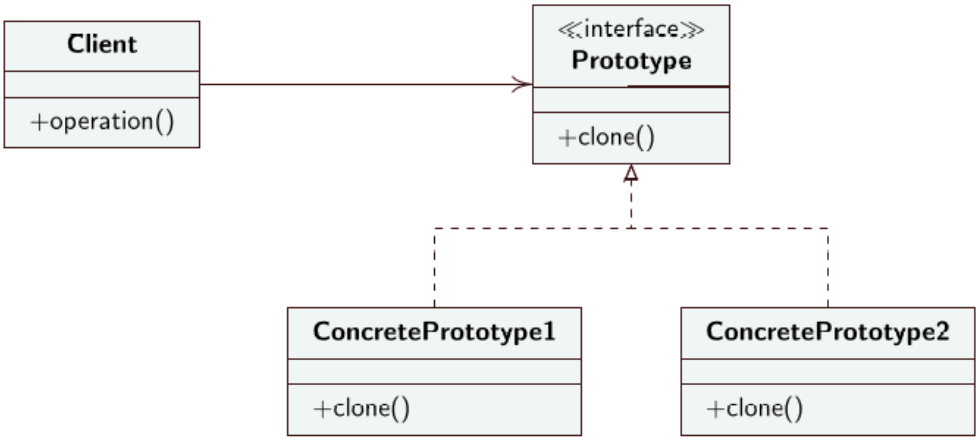


Diagramma UML del D.P. Prototype

- **Come funziona la registrazione dei Prototipi?** Si ha un **registro di prototipi**, ovvero una mappa (`nome -> istanza`), e può facilmente essere modificata mentre il programma è in esecuzione. Di registro ne esiste **uno solo**. Non appartiene né a Prototype né a ConcretePrototype. È una **classe separata**, solitamente nota come `PrototypeRegistry`. Internamente è come una mappa. Di solito questa classe è un **Singleton**, oppure viene passata come dipendenza. È il punto centrale da cui tutti i client pescano i prototipi.
- **Come fa il client a sapere quale voce gli interessa?** Il client conosce una **chiave semantica**, ovvero il nome logico di quello che vuole, ma non il nome della classe. Il client **sa cosa vuole funzionalmente**, ma non come è fatto internamente.
- **Perché servono più ConcretePrototype?** Perché ognuno poi implementa `clone()` in modo appropriato per sé stesso. Servono classi diverse perché

oggetti diversi hanno struttura e comportamento diversi. Un cerchio ha un raggio, un Quadrato ha un lato, etc.

- Nell'esempio precedente, sul Cerchio e sul Quadrato, loro sarebbero i ConcretePrototype che implementano Prototype. Per questo non possiamo permetterci di avere una sola classe ConcretePrototype, perché sennò dovremmo schiacciare tutti gli attributi di più classi distinte in una sola classe, e ciò ovviamente non va bene. Abbiamo più `ConcretePrototype` perché abbiamo più **tipi di oggetti strutturalmente diversi** che vogliamo poter clonare.

Conseguenze:

- Il client non conosce la classe usata, poiché il client usa l'interfaccia comune, quindi il **numero di dipendenze del client è ridotto**. Come conseguenza di ciò, il client può usare classi specifiche (sottoclassi), pur non conoscendone il nome.
- Solitamente, il codice che crea oggetti è fisso a compile-time. Così, invece, si può **registrare un nuovo prototipo** (o rimuovere un prototipo) **a runtime**. Ovviamente, possiamo anche costruire l'oggetto prototipo a runtime, per poi poterlo registrare come prototipo. Questo è possibile grazie al registro di prototipi di cui abbiamo parlato poco sopra.
- Si possono specificare nuovi oggetti con valori diversi, **senza creare nuove classi**. Senza Prototype, infatti, potremmo essere tentati di creare una classe per ogni possibile variante. Per esempio, classi come `CerchioRosso`, `CerchioBlu`, `CerchioGrande`, sono "sprecate". Conviene invece clonare un Cerchio e **cambiare l'attributo** dopo la clonazione. La variazione sta nello **stato**, non nella classe. **Evitiamo l'esplosione di sottoclassi**.
- **Prototipo con struttura composita:** un nuovo prototipo può essere usato per tenere una struttura diversa di oggetti (**composizione**). Gli oggetti della composizione sono attributi del prototipo, come in un **Composite**. Si avrà quindi come Prototipo un oggetto **composto da altri oggetti**. Per esempio un `Pannello` che contiene al suo interno `Bottoni`, `Label`, `TextField`. Questi sotto-oggetti sono attributi del prototipo. Clonando il prototipo otteniamo una copia dell'intera struttura composita in un colpo solo, senza doverla ricostruire pezzo per pezzo. Utile quando il Prototype collabora con il Composite.

- Pur avendo tante classi diverse da poter usare (i ConcretePrototype) non si ha la necessità di **implementare una gerarchia di Creator** (come per il Factory Method), in quanto un nuovo oggetto si ottiene tramite clonazione. Con Prototype, infatti, **aggiungiamo solo un'istanza al registro**, non una classe.

Prototype: Implementazione

- L'uso di un **manager di prototipi** (ovvero il registro di istanze visto poco prima) permette ai client di immagazzinare e recuperare i prototipi, prima di clonarli.
- L'implementazione dell'operazione di clonazione può essere la parte più difficile quando le strutture contengono **riferimenti circolari**. Il problema si verifica in situazioni come le seguenti: una classe `Dipendente` con attributo Manager, e una classe `Manager` con attributo lista dipendenti. Il **problema emerge durante la deep copy:** per clonare Dipendente dobbiamo clonare prima il suo Manager, ma per clonare il Manager dobbiamo prima clonare i suoi Dipendenti, che a loro volta richiedono di clonare il Manager, e così via. Per gestirlo correttamente bisogna tenere traccia degli oggetti già clonati durante la copia, e riusare il clone già creato invece di ricominciare. Parte più delicata dell'implementazione. Si usa una **mappa degli oggetti già clonati**.
- Sebbene un'operazione di clonazione potrebbe essere fornita dal linguaggio di programmazione, l'operazione di clonazione potrebbe effettuare una **copia superficiale (shallow copy)** anziché una **copia profonda (deep copy)**.
  In una shallow copy i riferimenti tenuti negli attributi dell'oggetto da clonare sarebbero condivisi fra originale e clone, e questo spesso non è sufficiente, quindi occorre una deep copy, dove ognuno degli oggetti ha effettivamente la propria copia personale degli attributi in questione.
- I cloni potrebbero necessitare di essere inizializzati. La classe prototipo potrebbe definire operazioni per impostare dati. Dato che il clone nasce con lo stesso stato del prototipo, **a volte alcune informazioni dovranno essere diverse per ogni clone**. Per esempio: il prototipo `Contratto`, template configurato con clausole standard. Lo cloniamo per creare un contratto reale, ma il clone ha ancora l'ID del prototipo, la data di creazione del prototipo, nome del cliente del prototipo e così via. Cioè, è ancora un oggetto "generico" e ci serve renderlo più "personale", aggiungendo dei dati che sono specifici per ogni clone. La classe ConcretePrototype, quindi, potrà definire metodi

appositi per questa **inizializzazione post-clonazione**. Questi metodi ovviamente potrebbero essere dichiarati e definiti anche in Prototype.

Esempio Prototype

Si vuole tenere un'agenda di impegni. Varie voci possono essere aggiunte all'agenda. Lo stato dell'agenda è un insieme di impegni presi. Una nuova istanza dell'agenda deve poter avere accesso agli impegni precedentemente inseriti.

In modo simile, si vogliono inserire note in un quaderno e accedere alle note precedenti quando si ha una nuova istanza di quaderno.

D.P. Adapter

Intento: converte l'interfaccia di una classe in un'altra interfaccia che i client si aspettano. Adapter permette ad alcune classi di interagire, eliminando il problema di interfacce incompatibili.

Problema:

- Alcune volte una classe di una libreria non può essere usata poiché incompatibile con l'interfaccia che si aspetta l'applicazione. Ovvero nome metodo, parametri, tipo parametri, di chiamate all'interno dell'applicazione che non sono corrispondenti a quelli offerti da una classe di libreria.
- Non è possibile cambiare l'interfaccia della libreria, poiché non si ha il sorgente (e comunque non conviene cambiarla).
- Non è possibile cambiare l'applicazione (il chiamante), e si può voler cambiare quale metodo chiamare, senza renderlo noto al chiamante.

Soluzione (versione Object Adapter):

- **Target:** è l'interfaccia che il chiamante si aspetta.
- **Client:** usa oggetti che sono conformi all'interfaccia Target.
- **Adaptee:** è l'oggetto di libreria che necessita l'adattamento.
- **Adapter:** converte, ovvero adatta, la chiamata che fa una classe client all'interfaccia della classe di libreria. Il chiamante usa l'Adapter come se fosse l'oggetto della libreria. La classe con ruolo **Adapter implementa Target, tiene il**

riferimento all'oggetto di libreria (Adaptee), e sa come invocarlo, ovvero chiama i metodi di Adaptee.

- Segue il meccanismo della **composizione**.

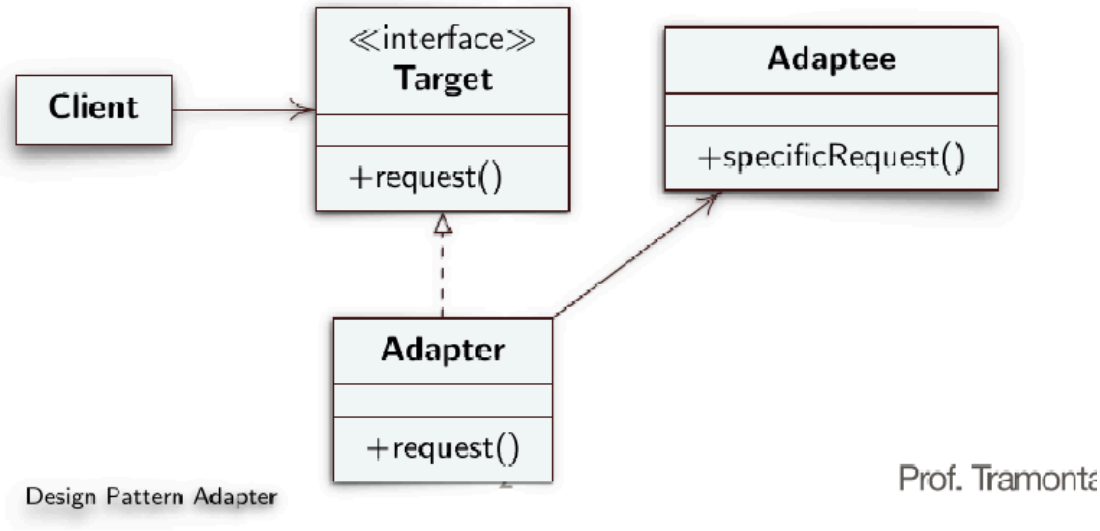


Diagramma delle classi dell'Adapter

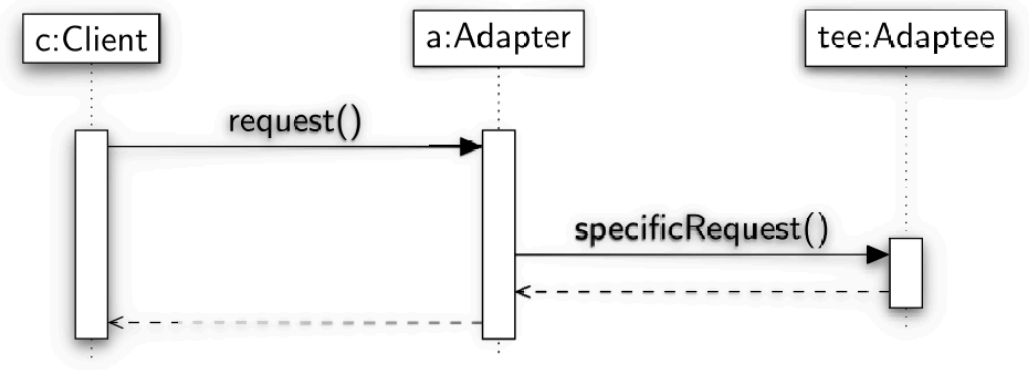


Diagramma di sequenza dell'Adapter

Soluzione (versione Class Adapter):

- Identica alla precedente, ma qui Adaptee è superclasse di Adapter, e quindi Adapter non tiene un riferimento a un oggetto di tipo Adaptee, ma si limita a chiamare i metodi della superclasse. La **differenza** è che non **posso scegliere a runtime quale Adaptee usare**, perché è già "hardcoded" nella gerarchia. Segue il meccanismo dell'**ereditarietà**.
- Una proprietà di questa versione è il **two-way Adapter**. Adapter espone entrambe le interfacce allo stesso tempo, in quanto implementa Target ed estende Adaptee, in quanto può essere passato a codice che si aspetta un Target e a codice che si aspetta un Adaptee. **Non adatta solo in una direzione**, ma può essere in mezzo a due sistemi che parlano linguaggi diversi e fare da ponte. Ovvero può essere passato sia a chi si aspetta Adaptee sia a chi si aspetta Target.

Conseguenze:

- Client e classe di libreria Adaptee rimangono indipendenti. Client, infatti, conosce solo Target, e Adaptee conosce solo sé stesso. Possiamo cambiare l'Adaptee modificando semplicemente il codice di Adapter, senza toccare il client, e viceversa.
- Il ruolo Adapter può cambiare il comportamento dell'Adaptee, in quanto può aggiungere **test di precondizioni e postcondizioni**. Precondizioni: cosa si deve soddisfare prima di eseguire. Postcondizioni: cosa è verificato se tutto è andato bene.
- L'Object Adapter può implementare la tecnica di **Lazy Initialisation**, ovvero si aspetta che avvenga un'invocazione a un metodo di Adaptee prima di istanziarlo. Alternativa al costruire l'Adaptee direttamente nel costruttore. Conviene come approccio perché potremmo sprecare risorse e non è detto che lo useremo.
- L'Adapter aggiunge un **livello di indirettezza**. Ogni invocazione del Client ne scatena un'altra fatta dal ruolo di Adapter. Possibile **rallentamento** (trascurabile), e codice più difficile da comprendere, quindi paghiamo un costo di **leggibilità**.
- Se Target e Adapter coincidessero, il Client si ritroverebbe direttamente accoppiato alla libreria, e quindi non favorirebbe il disaccoppiamento, e quindi porterebbe a una cascata di cambiamenti nel caso in cui la libreria venisse

aggiornata, in quanto dovremmo modificare Client, Target, e le eventuali sottoclassi di Target. Una modifica che sarebbe potuta essere locale e solo su Adapter si propaga invece su tutto il sistema.

- Target esiste proprio per essere un **contratto**, scollegato da qualsiasi implementazione specifica. Se Target e Adapter coincidessero, non potrei più cambiare Adapter facilmente, magari ne ho più di uno, e qualora facessi delle modifiche ad Adapter, queste si ripercuoterebbero subito sul Client. Non c'è più una classe/interfaccia, come faceva Target prima, che "assorbe" le modifiche e protegge il client da esse. Finché la firma di Target non cambia, qualsiasi cosa succeda dentro Adapter è **invisibile al client**.

Altro esempio di Diagramma:

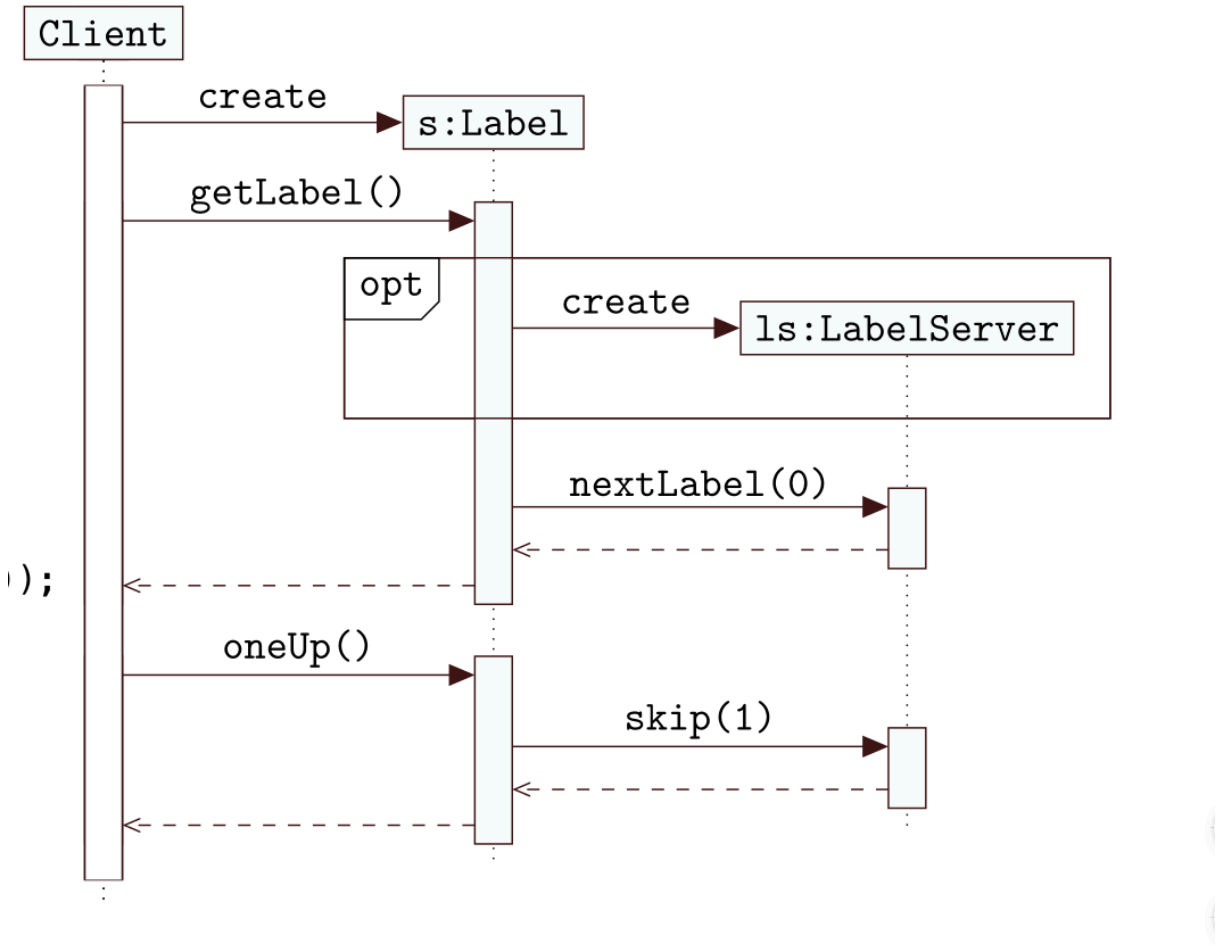


Diagramma UML di sequenza su un esempio specifico

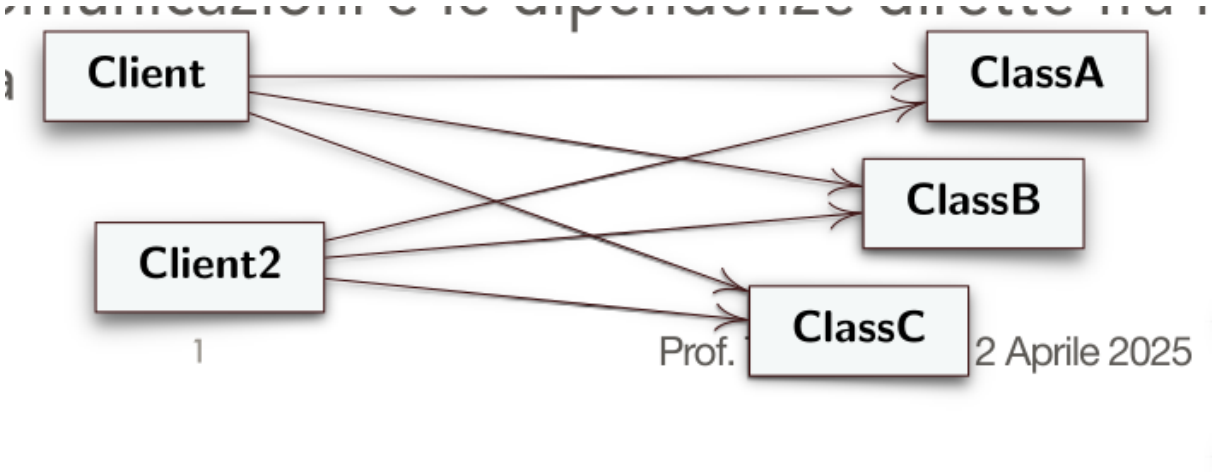
Nota: quest'ultimo diagramma è interessante in quanto ci mostra una notazione dell'UML di sequenza. Il rettangolo con opt in alto a sinistra indica un'operazione **opzionale (optional)**. Equivale a un if senza else. Per fare if/else si usa il rettangolo con **alt**, e per rappresentare i loop si usa **loop**.

D.P. Facade

Intento: fornire un'**interfaccia unificata** al posto di un insieme di interfacce in un **sottosistema** (consistente di un insieme di classi). Definire un'interfaccia di **alto livello** (semplificata) che rende il sottosistema più facile da usare.

Problema:

- Spesso si hanno tante classi che svolgono funzioni correlate e l'insieme delle interfacce può essere complesso.
- Può essere difficile capire qual è l'interfaccia essenziale ai client per l'insieme di classi.
- Si vogliono ridurre le comunicazioni e le dipendenze dirette fra i client ed il sottosistema.



La situazione in diagramma UML prima di usare il Facade

Soluzione:

- **Facade:** fornisce un'unica interfaccia semplificata ai client e nasconde gli oggetti del sottosistema, questo riduce la complessità dell'interfaccia e quindi

delle chiamate. Facade si occupa di invocare i metodi degli oggetti che nasconde.

- **Client:** interagisce solo con l'oggetto Facade.

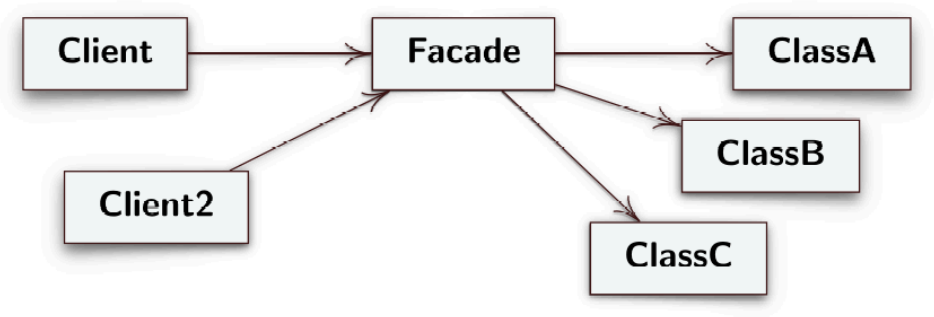


Diagramma UML del Facade

Conseguenze:

- **Nasconde ai client l'implementazione** del sottosistema.
- Promuove l'**accoppiamento debole** tra sottosistema e client. In questo modo, se un domani le classi del sottosistema dovessero subire modifiche, il client rimarrebbe invariato (isolato dai cambiamenti), e gli unici aggiustamenti sarebbero da fare sul Facade.
- Riduce le dipendenze di compilazione in sistemi grandi. Se si cambia una classe del sottosistema, si può ricompilare la parte di sottosistema fino al Facade, quindi non i vari client. Es. se cambia `English.h`, il compilatore ricompila tutti i file che lo includono direttamente. Con Facade, solo `Facade.cpp` include le classi del sottosistema, e quindi solo lui viene ricompilato, non tutti i client.
- Non prevenie l'uso di client più complessi, quando occorre, che accedono ad oggetti del sottosistema. Facade **non obbliga** nessuno a usare solo l'interfaccia semplificata. Se un client avanzato ha bisogno di fare qualcosa che Facade non espone, può comunque accedere direttamente alle classi del sottosistema. Facade **è una comodità, non un blocco**.

- **Implementazione:** per rendere gli oggetti del sottosistema non accessibili al client, le corrispondenti classi possono essere annidate dentro la classe Facade. Quindi, se vogliamo che i client **non possano mai accedere** direttamente al sottosistema, possiamo annidare le classi del sottosistema dentro Facade.
- Il Facade **non elimina la complessità:** la **sposta e la incapsula** in un posto solo. Il client ci guadagna in semplicità, e il sottosistema resta invariato.

D.P. Composite

Intento: comporre oggetti in strutture ad albero per rappresentare gerarchie di parti o del tutto. Composite permette ai client di trattare oggetti singoli e composizioni di oggetti uniformemente.

Problema:

- Immaginando di lavorare su un File System, abbiamo due tipi di oggetti: File e Cartelle. Senza Composite, il codice Client che vuole fare un'operazione (come `stampaContenuto`) deve distinguere i due casi: se l'elemento considerato è un file oppure una cartella.
- Supponendo di gestirla tramite if/else, se a un certo punto notassimo che l'elemento non è un file, significa che è una cartella. A questo punto, stampiamo il contenuto della cartella e poi procediamo a stampare il contenuto dei singoli elementi della cartella. Tuttavia, non sappiamo se i singoli elementi della cartella sono a loro volta cartelle o file, e quindi dovremmo scrivere altri **if/else annidati**, e chiaramente questo non è un approccio corretto per risolvere il problema.
- Vogliamo fare in modo che **file e cartelle (Leaf e Composite) implementino la stessa interfaccia**, e così il client non chiederà mai se l'elemento è un file o una cartella, e in questo modo **si può chiamare semplicemente** `stampaContenuto` su ciascuno. Che siano file o cartelle, non importa: è **ricorsione naturale**.

Motivazione:

- È necessario **raggruppare** elementi semplici tra loro per formare elementi più grandi.

- Se nell'implementazione c'è **distinzione** tra classi per elementi semplici e classi per contenitori di questi elementi semplici, il codice che usa queste classi deve trattarli in modo differente. Questa distinzione rende il codice più complicato.
- Composite permette di descrivere una composizione **ricorsiva**, in modo che i client non debbano fare distinzione tra tipi di elementi. I client tratteranno tutti gli oggetti della struttura uniformemente.

Esempio 1, i File:

- Le operazioni su disco permettono la gestione di file (immagini, testo, etc) e la gestione di cartelle. Esempi di operazioni sono: lettura, scrittura, esecuzione, creazione.
- Le classi client devono poter fare **operazioni su file e cartelle**, senza distinguere fra essi.
- Una cartella deve poter contenere al suo interno sia file che altre cartelle, queste a sua volta contengono altri elementi, e così via.

Esempio 2, le Figure:

- In un editor di figure ho sia figure semplici, come Linea, Box, etc, che figure composte, ovvero **raggruppamenti** di figure semplici e composte.
- Le classi client devono poter trattare allo stesso modo sia le figure semplici che quelle composte, quando avviano operazioni come disegna, sposta, etc.

Soluzione:

- **Component:** interfaccia che rappresenta elementi semplici e non. Si occupa di:
    1. **Dichiarare le operazioni** degli oggetti della composizione, ovvero le operazioni che il client chiamerà su tutti;
    2. **Implementare le operazioni comuni alle sottoclassi** in modo appropriato. Così se c'è qualcosa che Leaf e Composite fanno allo stesso modo, possiamo metterlo qui una volta sola;
    3. **Dichiarare le operazioni per l'accesso e la gestione degli elementi semplici**. Queste operazioni servono solo a Composite, ma stanno su

Component così permettono al client di usarle senza dover fare `instanceof`. Sono operazioni come `aggiungi()`, `rimuovi()`, `getFigli()` etc. A volte queste operazioni vanno messe direttamente sul Composite.

4. Può anche definire un'operazione che permette ad un elemento di **accedere all'oggetto padre** nella struttura ricorsiva.

Grazie a Component, **il client non deve sapere con chi sta parlando**.

- **Leaf:** classe che rappresenta **elementi semplici** (detti **child/children**). È sottoclasse di Component. Implementa il comportamento degli oggetti semplici, infatti implementa concretamente le operazioni dichiarate da Component. Operazioni come `aggiungi()` e `rimuovi()` o non le implementa, o le implementa lanciando un errore. Quando il client chiama `stampaContenuto()` su una Leaf, semplicemente esegue l'operazione e basta, senza nessun tipo di ricorsione.
- **Composite:** classe che rappresenta **elementi contenitori**. È sottoclasse di Component.
  1. Definisce il comportamento per l'aggregato di elementi child.
  2. Tiene il riferimento a ciascuno degli elementi child e **implementa operazioni per gestire elementi child**. Tiene una lista di Component, non di Leaf o di Composite specifici. Questo rende la struttura arbitrariamente profonda.
  3. Si occupa anche di **delegare operazioni**: quando il client chiama `stampaContenuto()` su un Composite, lui si occupa di iterare questa istruzione sui figli e di chiamare il metodo su ciascuno.

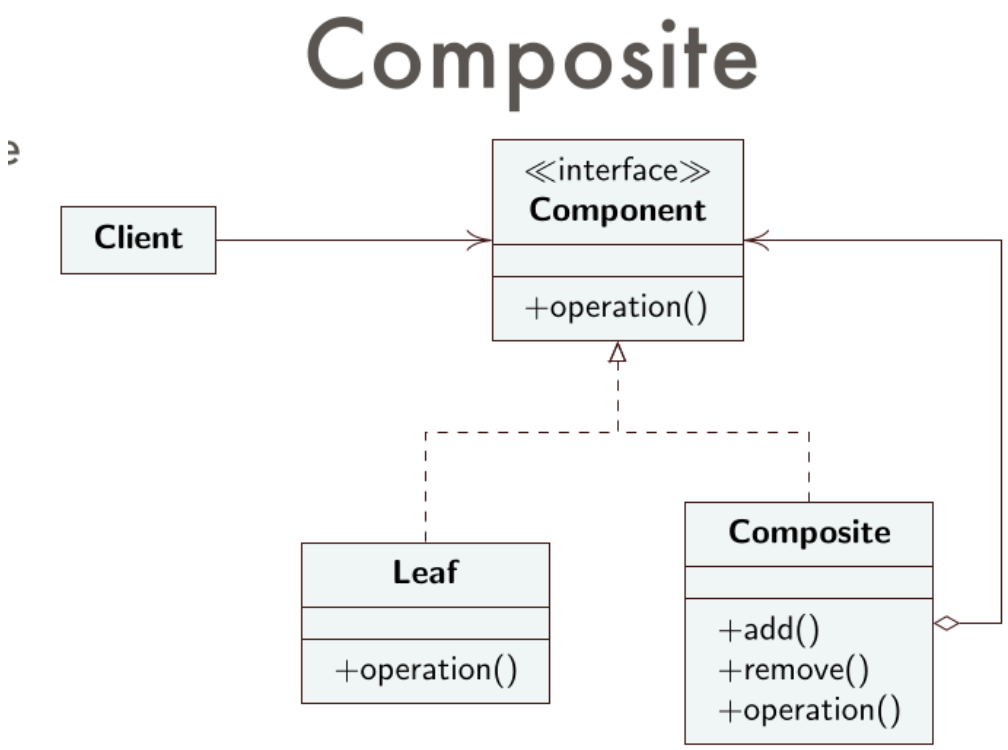


Diagramma UML delle classi del D.P. Composite

Tipi di relazioni:

- **Associazione:** rappresenta una relazione in cui una classe **usa** un'altra classe. Es. Client che usa Component.
- **Aggregazione:** rappresenta una relazione in cui una classe (**whole**) consiste di altre classi (**parti**). Ovvero, il **Composite aggrega (e consiste di) Leaf** (parti). Chi aggrega e le parti **esistono indipendentemente** l'uno dall'altro. Quindi le parti **hanno vita propria**. Si rappresenta con un **rombo vuoto** sul lato del whole. Nel nostro esempio, possiamo prendere un File e spostarlo in un'altra Cartella, non viene distrutto insieme al contenitore.
- **Composizione:** una forma di aggregazione in cui la parte non può esistere senza l'intero (whole). La parte appartiene all'intero ed è distrutta quando l'intero è distrutto. Si rappresenta con un **rombo pieno**. In generale, usiamo il rombo pieno se l'oggetto A possiede/contiene un oggetto B che senza l'oggetto A non avrebbe senso di esistere. Per esempio, uno Stato è **intrinsecamente legato** al suo Ordine, e Stato come informazione a sé stante,

senza il suo ordine, non ci dice niente. Se muore Ordine, muore Stato. Quindi siamo nel caso in cui B **non ha vita propria** fuori dal suo contenitore, c'è un **vincolo di composizione**.
Nota: A potrebbe tranquillamente continuare a esistere anche senza B.

Collaborazioni:

- I client usano l'interfaccia di Component per interagire con elementi della struttura composita. Se il ricevente del messaggio è Leaf, la richiesta è gestita direttamente. Se il ricevente è un Composite, questo invia la richiesta ai suoi child e possibilmente avvia operazioni addizionali prima e dopo.

Conseguenze:

- Elementi semplici possono essere composti in elementi più complessi, questi possono essere composti, e così via. **Composizione ricorsiva**.
- Un client che si aspetta un elemento semplice può riceverne uno composto.
- **I client sono semplici**, trattano strutture composte e semplici uniformemente. I client non devono sapere se trattano con un Leaf o con un Composite.
- Nuovi tipi di elementi (Leaf o Composite) possono essere aggiunti e potranno funzionare con la struttura e i client esistenti. Per esempio, quando aggiungiamo `Shortcut extends Component`, ci basta implementare `operation()` al suo interno, e il client esistente **non va toccato**. Il client così riuscirà a chiamare operation() su Shortcut esattamente come su File o Cartella, senza nemmeno sapere che Shortcut esiste. Segue il principio **Open/Closed:** aperto all'estensione (aggiungendo nuovi tipi), ma chiuso alla modifica (il codice client esistente non cambia mai). Tutto il codice presente in Leaf e Composite non verrà modificato per l'aggiunta di Shortcut, in quanto abbiamo progettato bene il sistema, grazie all'utilizzo del Design Pattern.
- Non è possibile a **design-time** vincolare il Composite solo su certi elementi Leaf (in base al tipo), invece dovranno essere fatti dei controlli a runtime.
  Esempio: supponiamo di voler fare una Cartella che contiene solo File e non altre Cartella. Non possiamo esprimerlo nella gerarchia delle classi, in quanto Composite tiene una lista di Component generici, e quindi sarebbe necessario fare un **controllo a runtime**.

Ulteriori dettagli:

- Per facilitare la navigazione degli oggetti, gli elementi child (ovvero **qualsiasi nodo che ha un padre**) mantengono il riferimento all'oggetto che li contiene. Il riferimento è inserito in Component, mentre Leaf e Composite possono implementare le operazioni per gestirlo.
- Per avere client che non distinguono se trattano con Leaf o Composite (è uno degli obiettivi), la classe Component dovrebbe **definire quante più operazioni in comune possibile**. Le classi Leaf e Composite faranno override delle operazioni.
- Dove implementare le **operazioni di gestione di child,** `add()` **e** `remove()`**?**
    1. Se dichiarate in Component si ha **trasparenza**, ma per le classi Leaf tali operazioni non hanno significato. La **sicurezza** nell'uso è quindi compromessa, poiché i client potrebbero avviarle su Leaf. Se i client avviano un'operazione add() su Leaf si ignora la richiesta, questo potrebbe indicare un difetto del programma.
    2. Se dichiarate in Composite si ha **sicurezza**, poiché a compile time si verifica che tali operazioni non possono essere chiamate su Leaf, ma si perde la **trasparenza**.
- Si può inserire una operazione `getComposite()` in Component che ritorna null e che è ridefinita in Composite per ritornare il riferimento a sé stesso. I client dovrebbero comunque distinguere il tipo di risultato e fare operazioni differenti, **niente trasparenza**. Stiamo semplicemente sostituendo `instanceof` con un controllo sul `null`.
- La lista che contiene elementi child deve essere definita in Composite, altrimenti se fosse definita in Component **si sprecherebbe spazio**, poiché ogni Leaf avrebbe tale variabile anche se non deve usarla mai.
- L'**ordinamento di child** per un Composite potrebbe essere importante e va tenuto in considerazione su certe implementazioni.
- Il Composite potrebbe implementare una **cache**, per ottimizzare le prestazioni. È utile quando si implementa un'operazione che deve ricercare tra tutti i suoi componenti child, come per esempio un metodo che somma la dimensioni di tutti i child. I componenti child devono poter accedere ad un'operazione che permette di invalidare la cache del Composite, in quanto se qualcuno di loro

cambia qualcosa, la cache diventa **obsoleta**. Serve un'operazione del tipo `invalidaCache()` che risale al padre e gli dice: "il tuo valore cached non è più valido, ricalcolalo la prossima volta". La propagazione deve arrivare **fino alla radice**. Ovviamente ci si ferma quando il padre è uguale a null.

D.P. Decorator

Intento: aggiungere ulteriori **responsabilità ad un oggetto dinamicamente**. I decorator forniscono un'alternativa flessibile all'implementazone di sottoclassi per estendere funzionalità.

Motivazioni:

- Alcune volte si vogliono **aggiungere responsabilità a singoli oggetti**, e non all'intera classe. Per aggiungere responsabilità si potrebbe far uso dell'ereditarietà, tuttavia non si avrebbe flessibilità: un client non può controllare come e quando decorare un componente, in quanto la decisione è stata presa a **design-time**.
- Un approccio più flessibille è racchiudere (**wrap**) il componente in un altro oggetto che aggiunge una responsabilità. L'oggetto **wrapper è un decorator**. Il decorator è conforme all'interfaccia che decora, quindi **la sua presenza è trasparente agli oggetti che usano il componente**. Invece di modificare un oggetto esistente, lo avvolgiamo in un secondo oggetto che lo contiene come campo interno.
- Le **responsabilità possono essere sottratte dinamicamente**. I decorator possono essere **annidati ricorsivamente**, per aggiungere più responsabilità.
- A volte la **creazione di sottoclassi non è praticabile**. Un numero grande di estensioni produrrebbe un numero enorme di sottoclassi per gestire tutte le combinazioni. Usando l'ereditarietà avremmo bisogno di un numero enorme di sottoclassi per poter gestire tutte le combinazioni possibili.

Esempio:

- Si vuole aggiungere la proprietà Bordo ad un componente Testo. **Se si eredita Testo dalla classe Bordo**, gli oggetti della sottoclasse avranno questa proprietà, ma **non si avrà flessibilità**: non si possono avere oggetti senza tale proprietà.

- Alternativa flessibile, inserire il componente Testo dentro un oggetto Bordo.
- L’oggetto che racchiude, chiamato DecoratorBordo, manda la richiesta al componente Testo e aggiunge altre attività prima o dopo l’invio della richiesta.
- Il **Decorator non sostituisce** il comportamento del componente: lo delega e poi aggiunge qualcosa intorno.

Soluzione:

- **Component:** definisce l’interfaccia per gli oggetti che possono avere aggiunte le responsabilità dinamicamente. Questa interfaccia è il contratto che devono rispettare sia gli oggetti reali che i loro decorator. Grazie a ciò, il client non sa mai con chi sta parlando: se con un oggetto non wrappato o wrappato.
- **ConcreteComponent:** definisce un oggetto su cui poter aggiungere responsabilità. È l’oggetto che vogliamo decorare, e non sa niente dei decorator che lo avvolgeranno. Si occupa di implementare Component.
- **Decorator:**
  1. Mantiene un riferimento a un oggetto Component, quindi può wrappare qualsiasi cosa implementi Component, sia un ConcreteComponent che un altro Decorator.
  2. Si occupa anche di implementare Component, e quindi dall’esterno è indistinguibile da un ConcreteComponent, dato che implementano la stessa interfaccia.
  3. Decorator inoltra le richieste al suo oggetto Component e può fare altre operazioni prima e dopo l’inoltro della richiesta.
- **ConcreteDecorator:** implementa la responsabilità aggiunta al Component. Può decorare un oggetto in due modi: aggiungendo dati (stato) o aggiungendo logica (comportamento).

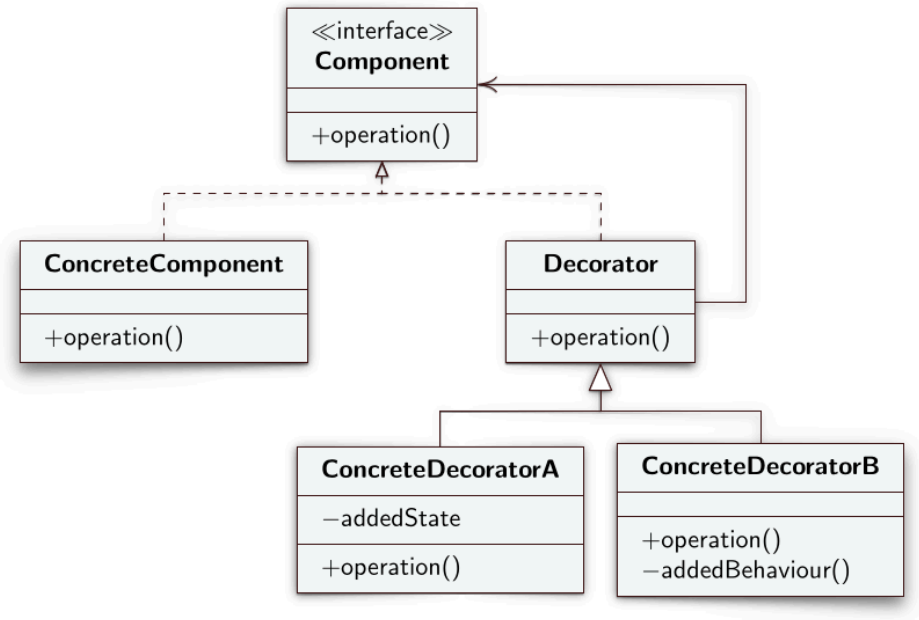


Diagramma UML delle classi di Decorator

Doppia relazione di Decorator verso Component: si occupa sia di implementarlo che di contenerne uno. Lo implementa cosicché il client lo vede come un Component normale, e quindi garantisce **trasparenza**. Ne contiene uno perché così può wrappare qualsiasi Component, anche un altro Decorator (**annidamento**). Se non ne contenesse uno, non avrebbe modo per wrappare questi Component. Inoltre, poiché Decorator implementa Component, soddisfa quel tipo, e quindi nel campo component di Decorator si può mettere qualsiasi cosa: un ConcreteComponent o un ConcreteDecorator.

Conseguenze:

- **Più flessibilità** rispetto all'ereditarietà poiché si possono aggiungere responsabilità dinamicamente.
- La stessa responsabilità può essere aggiunta più volte, semplicemente aggiungendo due istanze dello stesso ConcreteDecorator.
- Prevedere per le classi in alto nella gerarchia tutte le responsabilità che servono significherebbe avere per esse **troppe responsabilità**. Mettere tutto in ConcreteComponent o in Component ci porta ad avere classi gonfie e che

fanno troppe cose, violando il principio di Singola Responsabilità. ConcreteDecorator sono invece indipendenti e permettono di aggiungere responsabilità successivamente.

- **Svantaggio:** l'identità (tipo e riferimento) del ConcreteDecorator non è quella del ConcreteComponent, quindi **non si dovrebbero confrontare i riferimenti**. Quando wrappiamo un ConcreteComponent dentro un ConcreteDecorator, l'oggetto risultante non è più un ConcreteComponent, e quindi se in qualche punto del codice facciamo confronti basati sul tipo o sul riferimento, il Decorator rompe quella logica.
- Si avranno tanti piccoli oggetti, che differiscono nel modo in cui sono interconnessi. Questo perché una catena di decorator produce tanti oggetti distinti in memoria, ognuno che contiene il riferimento al precedente. Ciò rende il **debug più difficile**, e infatti se qualcosa va storto bisogna seguire la catena di deleghe per capire dove non funziona. Inoltre, la **comprensione del sistema è più difficile**: un sistema con molti decorator annidati può diventare difficile da leggere se non si conosce il pattern.

D.P. Bridge

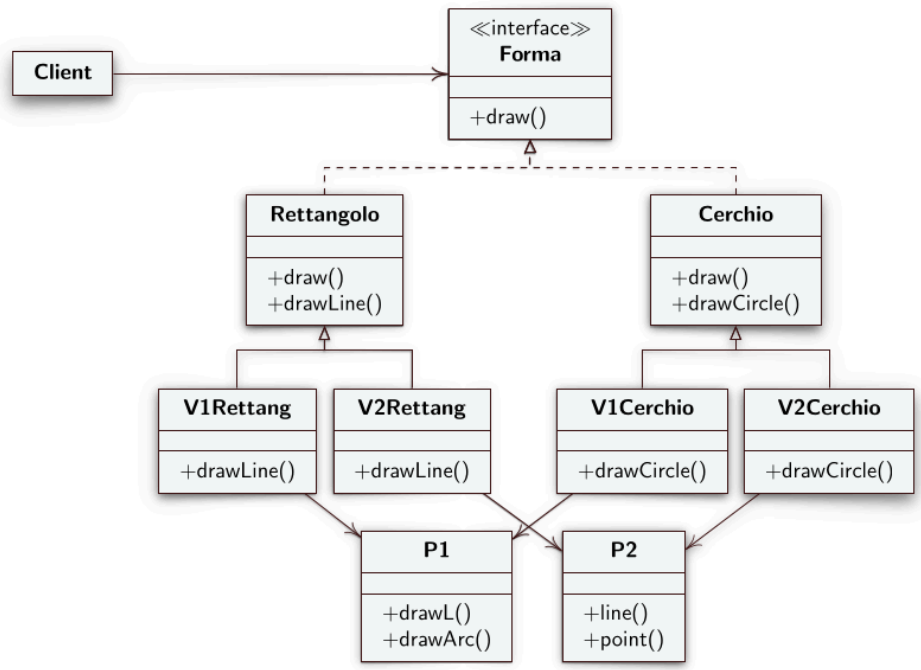
Intento: disaccoppiare una astrazione dalla sua implementazione così che le due possano variare indipendentemente.

Motivazione:

- Quando una astrazione può avere varie implementazioni, di solito si usa l'**ereditarietà**. Una classe astratta definisce l'interfaccia dell'astrazione, le sottoclassi concrete la implementano in modi diversi.
- Questo approccio **non è flessibile**, poiché **collega l'astrazione all'implementazione permanentemente**, e rende difficile modificare, estendere e usare astrazioni e implementazioni indipendentemente. Le conseguenze dell'ereditarietà sarebbero: esplosione combinatoria (con M forme e N piattaforme, necessito di M $*$ N classi concrete) e accoppiamento rigido: un'istanza di V1Rettang non può mai usare P2 a runtime, la piattaforma è decisa staticamente, a compile-time, e quindi non possiamo cambiare implementazione senza cambiare classe.

Esempio:

- In un sistema, occorre avere Rettangoli e Cerchi (astrazioni). Inoltre, occorre che Rettangoli e Cerchi siano disponibili per due piattaforme P1 e P2 (implementazioni).
- La piattaforma P1 mette a disposizione drawL() per disegnare linee, mentre la piattaforma P2 fornisce line().
- Struttura delle classi iniziale (prima del Bridge):



Considerazioni:

- La soluzione appena vista porterebbe a una **proliferazione di classi**.
- Se introducessi un'altra piattaforma P3 (implementazione) avrei bisogno di 6 classi concrete (2 forme * 3 piattaforme). Se introducessi un'altra forma (astrazione), avrei bisogno di 9 classi concrete (3*3).
- Per ogni variazione da introdurre vorrei invece un incremento lineare del numero di classi. Inoltre, presa una classe, per esempio la classe V1Rettang, essa è legata a una certa piattaforma P1 in modo permanente, ovvero una istanza di V1Rettang non può usare piattaforma diversa da P1. Vogliamo cambiare questo comportamento.

Idea del Bridge:

- Invece di fondere le due dimensioni (forme e piattaforme) in un'unica gerarchia, il Bridge le separa in **due gerarchie distinte**, e le collega attraverso un riferimento (la composizione).
- Forma tiene un riferimento a DrawingAPI (interfaccia implementata dalle piattaforme), e così il numero di classi scende da M*N a M+N.
- **Astrazione:** non intendiamo classe astratta nel senso Java. Significa la parte del sistema che definisce **cosa vogliamo fare**, ovvero l'interfaccia ad alto livello che il client usa. Nel nostro esempio è `draw()`.
- **Implementazione:** non si intende implementazione concreta nel senso Java. Significa la parte che definisce **come farlo a basso livello**, ovvero i "mattoncini" su cui l'astrazione si appoggia. Nel nostro esempio è `drawL()` e `line()`.
- Grazie al Bridge, possiamo:
  1. Aggiungere una forma senza toccare il codice delle piattaforme.
  2. Aggiungere una piattaforma senza toccare il codice delle forme.
  3. **Cambiare implementazione a runtime**, perché il riferimento è un campo dell'oggetto, non qualcosa di intrinseco alla gerarchia. Ciò sarebbe impossibile con l'ereditarietà pura: se abbiamo `V1Rettang`, non possiamo farlo diventare un `V2Rettang` a runtime.

Soluzione:

- **Abstraction**: definisce l'interfaccia per i client e mantiene un **riferimento a un oggetto di tipo Implementor**. Abstraction inoltra le richieste del client al suo oggetto Implementor. Abstraction **non è un'interfaccia**. In sostanza, si occupa di:
  1. Dichiarare operazioni ad alto livello che il Client può chiamare;
  2. Tenere al suo interno un campo di tipo Implementor. Quando `operation()` viene chiamato, internamente delega il lavoro basso-livello a `impl.operationImpl()`.

- **RefinedAbstraction:** estende l'interfaccia definita da Abstraction. Possono aggiungere operazioni specifiche, ma continuano ad appoggiarsi al campo ereditato da Abstraction. Per esempio, Rettangolo può definire un metodo specifico che chiama `impl.operationImpl()` più volte, magari per disegnare i quattro lati.
- **Implementor:** definisce l'interfaccia per le classi dell'implementazione. Questa interfaccia **non deve corrispondere esattamente ad Abstraction**, di solito Implementor **fornisce operazioni primitive**, mentre Abstraction definisce operazioni di più alto livello basate su tali primitive. Implementor **è un'interfaccia**.
- **ConcreteImplementor**: implementa l'interfaccia di Implementor e fornisce le operazioni concrete.

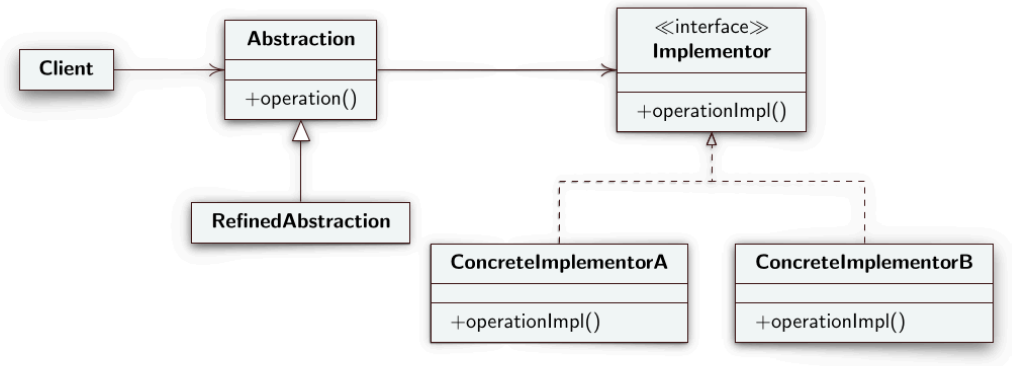


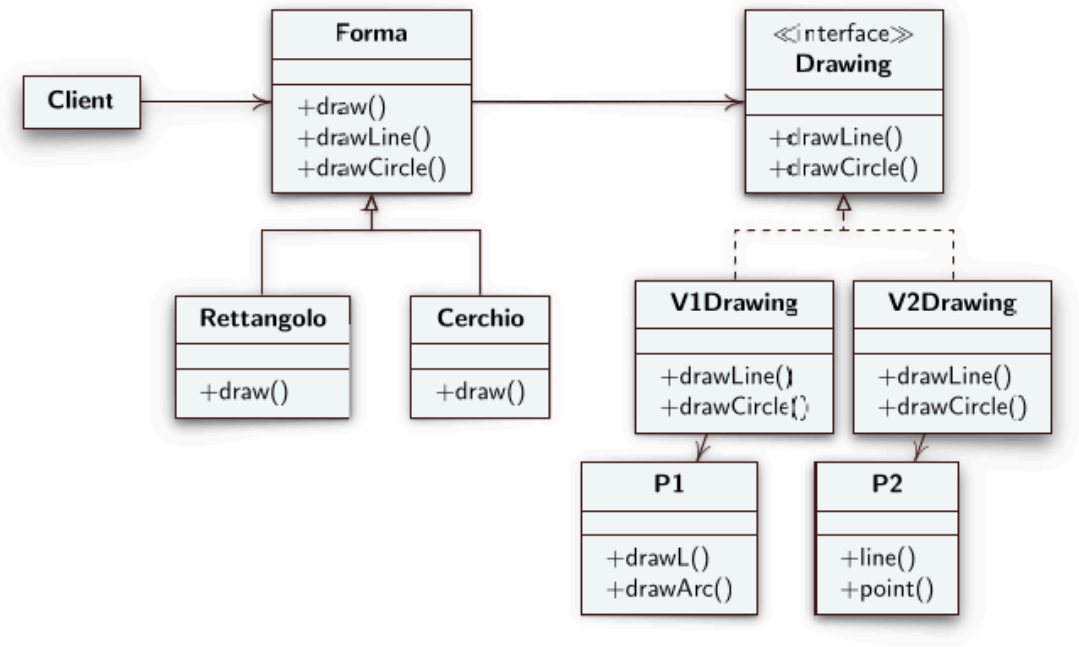
Diagramma UML delle classi del D.P. Bridge

Conseguenze:

- Bridge permette a una implementazione di **non essere connessa permanentemente a una interfaccia**, l'implementazione può essere configurata e anche cambiata a runtime.
- Il disaccoppiamento permette di cambiare l'implementazione **senza dover ricompilare Abstraction e i Client**, in quanto Abstraction dipende dall'interfaccia Implementor, e non dalle classi concrete.

- Solo certi strati del software devono conoscere Abstraction e Implementor. Il client, per esempio, conosce solo Abstraction (non conosce e non deve conoscere Implementor). Ciò favorisce l'**information hiding**.
- Notiamo che a sinistra c'è la gerarchia delle Astrazioni, e a destra la gerarchia delle Implementazioni.
- Le gerarchie di Abstraction e Implementor possono evolvere in modo indipendente. Possiamo infatti aggiungere una nuova RefinedAbstraction senza il bisogno di aggiungere un nuovo ConcreteImplementor, e viceversa.
- Il Bridge è la risposta pratica a una regola generale del design OO: **preferiamo la composizione all'ereditarietà**. Con l'ereditarietà diciamo **B è un A**, mentre con la composizione diciamo che **B ha un A**, e questo lo preferiamo in quanto è una relazione **flessibile**. Le variazioni delle due gerarchie sono **ortogonali**, ovvero completamente indipendenti le une dalle altre. Le variazioni per la parte di Abstraction riguarda le forme, e quindi sono variazioni legate a motivi geometrici/logici. Le variazioni per la parte di Implementor riguardano le piattaforme, che variano per motivi tecnici (es. aggiungo P3 perché ho un nuovo sistema operativo). Questi motivi di variazione non hanno niente a che fare l'uno con l'altro, e per questo si dicono ortogonali.

Struttura delle classi del nostro esempio dopo l'uso del Bridge:



- P1 e P2 rappresentano le **piattaforme già esistenti** (es. librerie di sistema, SDK grafici, etc). Hanno i loro metodi con nomi e firme fissati da altri:

  1. P1 espone drawL() e drawArc();
  2. P2 espone line() e point().

  Non possiamo modificarle: semplicemente esistono, dato che sono piattaforme già esistenti con metodi già pronti (magari scritti persino da noi), e i ConcreteImplementor si appoggiano a questi.

- L'interfaccia Drawing (Implementor) si aspetta drawLine() e drawCircle(). Le piattaforme reali P1 e P2 hanno metodi con nomi diversi, e quindi dobbiamo adattare P1 e P2 all'interfaccia che il Bridge si aspetta.

- V1Drawing e V2Drawing sono i ConcreteImplementor del Bridge, e internamente fanno da **wrapper** verso le piattaforme reali. Sono come degli adattatori delle piattaforme reali. Da fuori (ovvero da Forma) esistono solo V1Drawing e V2Drawing, che parlano il linguaggio di Drawing, e P1 e P2 sono completamente nascosti.

- **Nota importante:** possiamo notare l'utilizzo del Design Pattern **Adapter**. Infatti, V1Drawing svolge il ruolo di Adapter, Drawing il ruolo di Target e P1 il ruolo di

Adaptee. Il client dell'adapter, in questo contesto, è Forma, che vede solo Drawing (cioè Target). Adapter infatti risolve il problema che le piattaforme reali non parlano il linguaggio che il Bridge si aspetta, e quindi bisogna adattarle.

- Usare P1 o P2 internamente è un dettaglio implementativo di V1Drawing e V2Drawing: il Bridge non ne sa nulla e non gli interessa.

Sintesi:

- Il Bridge serve a separare un'astrazione dalla sua implementazione, in modo che le due possano evolvere e variare indipendentemente l'una dall'altra. Il problema che risolve è quello dell'esplosione di classi che si avrebbe usando solo l'ereditarietà. Se ho M forme e N piattaforme e uso l'ereditarietà per modellare entrambe le dimensioni insieme, finisco con M*N classi concrete, e in più ogni classe resta legata staticamente a una piattaforma, e quindi **non potrei cambiarla a runtime**.
- L'idea del Bridge è quella di spezzare questa unica gerarchia in due gerarchie separate, una per l'astrazione (cosa voglio fare, l'interfaccia ad alto livello) e una per l'implementazione (come lo faccio concretamente, a basso livello), e di collegarle non con l'ereditarietà ma con la composizione: l'astrazione tiene un riferimento all'implementazione. Così il numero di classi scende da M*N a M+N, e quel riferimento essendo un semplice campo dell'oggetto può essere cambiato anche a runtime.
- **Preferiamo la composizione all'ereditarietà** (più flessibile). Ovvero, preferiamo **costruire un oggetto includendo riferimenti ad altri oggetti**, e non semplicemente costruire un oggetto ereditando da una classe.

D.P. Observer

Design Pattern **comportamentale**, noto anche come **Publisher-Subscribe**.

Intento: definire una **dipendenza uno a molti** fra oggetti, così che quando un oggetto cambia stato tutti i suoi oggetti dipendenti sono notificati e **aggiornati automaticamente**.

Problema:

- Senza Observer, se abbiamo un oggetto Dati e tre viste (Tabella, Grafico, Monitor), dovremmo scrivere ogni volta `tabella.aggiorna()`, `grafico.aggiorna()`, `monitor.aggiorna()`. Dati è strettamente accoppiato a tutte le viste. Se aggiungiamo una quarta vista, dobbiamo modificare Dati. Se rimuoviamo Grafico, bisogna modificare Dati. Le classi **non sono più riusabili indipendentemente**.
- L'intento dell'Observer è quello di risolvere esattamente questo problema: Dati non deve sapere chi dipende da lui. Deve solo sapere che il suo stato è cambiato e **notificare chiunque si sia registrato**.

Motivazioni:

- Un sistema che è stato partizionato in un insieme di classi che cooperano deve mantenere la **consistenza** fra oggetti che hanno relazioni.
- Es. uno strumento di **presentazione** è separato dai **dati** dell'applicazione. Le classi che tengono i dati e quelle di presentazione sono **separate** e riusabili. Uno spreadsheet e un diagramma sono dipendenti dall'oggetto che contiene i dati e dovrebbero essere notificati dei cambiamenti dei dati.

Spiegazioni:

- L'Observer descrive come stabilire relazioni fra oggetti dipendenti.
- Gli oggetti chiave sono **Subject** e **Observer**.
- Un **Subject può avere tanti Observer** che dipendono da esso e gli Observer sono notificati quando lo stato del Subject cambia.
- **Meccanica della registrazione**: il Subject mantiene una lista di Observer registrati. Questo è il motivo per cui si chiama anche **Publisher-Subscribe**. Gli Observer si **iscrivono** (`attach`) al Subject quando vogliono ricevere notifiche. Gli Observer si **disiscrivono** (`detach`) quando non sono più interessati. Quando lo stato cambia, il Subject chiama `notify()` su tutti gli iscritti.
- Il Subject non sa nulla di chi siano i suoi Observer: li tratta tutti come oggetti che implementano l'interfaccia Observer. Questa è la ragione del **disaccoppiamento**.

Applicabilità:

- Un’astrazione ha due aspetti (es. dati e presentazione), ciascuno dipendente dall’altro. **Incapsulare questi aspetti** in oggetti separati permette di **riusarli indipendentemente**. Se mettiamo dati e presentazione insieme, nella stessa classe, allora non potremo avere due presentazioni diverse degli stessi dati, e non potremo riusare il modello dati con un’altra vista.
- **Un cambiamento** su un oggetto **richiede il cambiamento di altri**, non si conosce quanti oggetti sarà necessario cambiare. Si ha **scalabilità a runtime**: potrebbero esserci 1 o 100 Observer. Aggiungiamo e rimuoviamo dipendenze dinamicamente senza toccare il Subject.
- Un oggetto deve notificare altri oggetti senza fare assunzioni su chi sono tali oggetti, quindi gli oggetti non devono essere strettamente accoppiati. Promuove il **disaccoppiamento debole:** possiamo aggiungere un nuovo ConcreteObserver che non esisteva quando avevamo scritto il Subject, e funzionerà benissimo lo stesso.

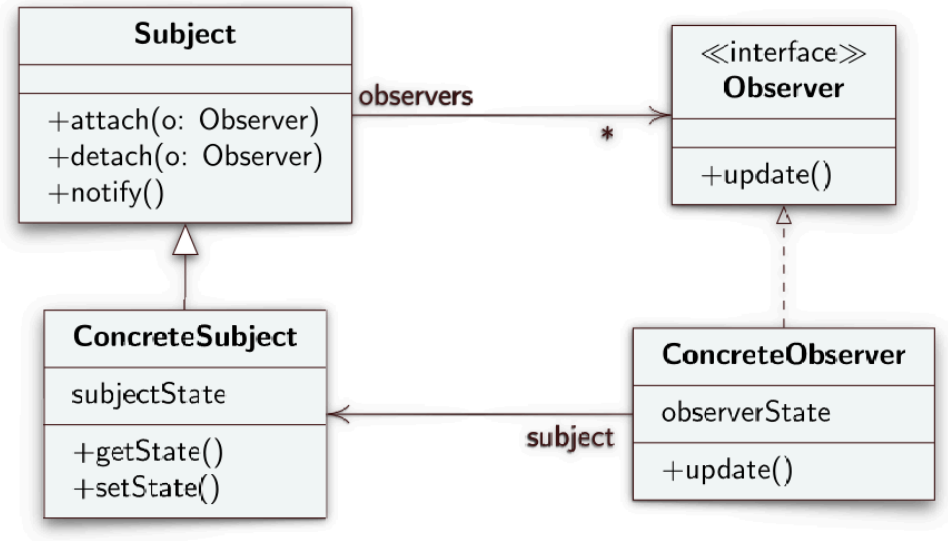
Soluzione:

- **Subject:** conosce i suoi osservatori, un qualsiasi numero di oggetti Observer può osservare un Subject. Implementa le operazioni per aggiungere e togliere oggetti Observer e per notificarli.
- **Observer:** definisce una interfaccia (operazione `update()`) comune a tutti gli oggetti che necessitano la notifica.
- **ConcreteSubject:** tiene lo stato che interessa agli oggetti ConcreteObserver. Notifica i suoi osservatori quando il suo stato cambia, grazie all’operazione `notify()` che eredita da Subject. Con notify() il Subject itera sulla sua lista e chiama update() su ciascun Observer. Lo stato del ConcreteSubject viene modificato dal Client attraverso il metodo `setState()`. Invece, `getState()` è quello che gli Observer usano per interrogare il Subject nel modello pull.
- **ConcreteObserver:** tiene un riferimento all’oggetto ConcreteSubject, e tiene lo stato che deve rimanere consistente con quello del Subject. Infatti, per questo motivo ha un campo `observerState` che deve rispecchiare il `subjectState`. È questo il problema di consistenza che il pattern deve risolvere. Implementa Observer per ricevere notifiche dei cambiamenti del Subject. Dopo la notifica può interrogare il ConcreteSubject per ottenere il nuovo dato.

Flusso dell'Observer:

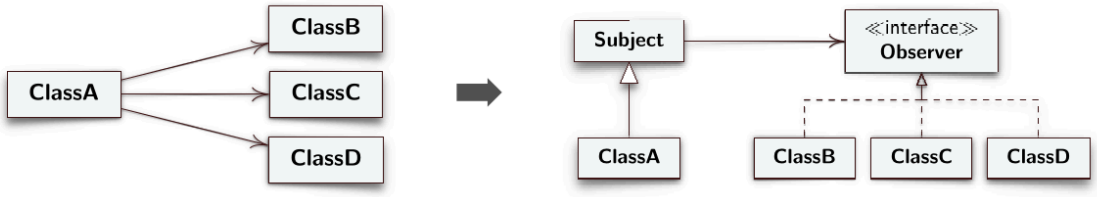
- Quando lo stato dell'oggetto ConcreteSubject cambia, l'oggetto ConcreteSubject chiama notify(), che chiamerà update() sugli oggetti ConcreteObserver per aggiornarli. Chi chiama notify()? Di questo se ne occupa setState() del ConcreteSubject, che chiama notify() internamente.

- Diagramma delle classi



Qual è il **significato dell'asterisco (*)?** L'asterisco in UML indica la **molteplicità**. Ovvero, indica quante istanze di un tipo sono associate a un'istanza dell'altro tipo. Ovvero, un Subject è associato a **zero o più** Observer.

- In generale, la situazione prima e dopo l'uso di Observer è la seguente:



Conseguenze:

- Il Subject conosce solo la classe Observer e non ha bisogno di conoscere le classi ConcreteObserver. ConcreteSubject e ConcreteObserver non sono accoppiati, quindi sono più facili da riusare e modificare. **Loose coupling**. La conseguenza è che possiamo aggiungere un nuovo ConcreteObserver senza toccare una riga di Subject o ConcreteSubject: principio **Open/Closed**.
- La notifica inviata da un ConcreteSubject è mandata a tutti gli oggetti che si sono iscritti, e il ConcreteSubject non sa quanti sono. Gli osservatori possono essere rimossi in qualunque momento. L'osservatore sceglie se gestire o ignorare la notifica. Fenomeno di **broadcast:** la notifica non è indirizzata a nessuno in particolare, ma a tutti gli iscritti.
- **Conseguenza negativa:** l'aggiornamento da parte del Subject può far avviare tante operazioni sugli Observer e altri oggetti per gli aggiornamenti. I cosiddetti **unexpected updates**.
- Nel modello pull, la notifica non dice agli Observer cosa è cambiato nel ConcreteSubject: è un evento che indica il completamento di un'operazione del ConcreteSubject.

Avvertenze:

- Con tanti Subject e pochi Observer, anziché tenere riferimenti a Observer nei Subject, si può usare una sola **tabella associativa** (in comune fra i Subject) per ridurre lo spazio impegnato. Si tratta di **ottimizzazione**.

- Un Observer potrebbe osservare più oggetti, per sapere chi ha mandato la notifica, come parametro di update(), si manda il riferimento al Subject. Quindi si passa `this` come parametro di `update()`.
- Subject chiama **notify()** dopo un cambiamento, oppure **aspetta un certo numero di cambiamenti**, in modo da evitare continue notifiche agli Observer. Quindi c'è l'approccio **automatico**, dove setState() chiama notify internamente a ogni modifica: overhead inutile se facciamo tanti setState di fila. Approccio **manuale** dove il client chiama notify() esplicitamente dopo tutti i setState. Più controllo, ma c'è il rischio di dimenticarsi di farlo.
- Per mantenere la consistenza fra Observer e Subject, un cambiamento dell'Observer deve essere comunicato al Subject (con setState()). Il Subject deve essere in uno **stato consistente prima di chiamare notify()**. Se lo chiamiamo nel mezzo di un aggiornamento parziale, si verificheranno problemi.
- Quando si vuole eliminare un Subject, gli Observer dovrebbero essere avvisati per cancellare il loro riferimento al Subject. Il Subject dovrebbe chiamare `detach()` su tutti i suoi Observer. Prima si distruggono gli Observer, poi il Subject, per evitare problemi di **zombie observer**.
- Il Subject può passare ulteriori informazioni quando chiama update(), e questo è il **modello push**. Nel modello **pull**, aspetta che l'Observer legga lo stato.
- Gli Observer che si registrano possono specificare gli **eventi di interesse**, in modo che con l'update() si manda solo ciò che è cambiato.

Vi sono **due varianti** di questo Design Pattern:

1. **Push model:** il Subject invia i dati aggiornati direttamente nella chiamata a `update()`.
   Pro: gli Observer non devono interrogare il Subject.
   Contro: il Subject deve sapere di quali dati hanno bisogno gli Observer. Il Subject dovrà quindi fare **assunzioni sui loro bisogni**.
2. **Pull model:** il Subject notifica solo che qualcosa è cambiato, e gli Observer interrogano il Subject per ottenere i dati.
   Pro: gli Observer prendono solo quello che serve.

Contro: richiedono un riferimento al Subject.
Questo è il modello **più flessibile** e anche il **più usato** nella pratica.

Sintesi:

- L'Observer è un pattern comportamentale, anche noto come Publisher-Subscriber. Il suo intento è definire una **dipendenza uno-a-molti** fra oggetti, in modo che quando un oggetto cambia stato, tutti i suoi dipendenti vengano notificati e aggiornati automaticamente, senza che il primo oggetto sappia esplicitamente chi sono.
- Supponiamo di avere dei dati e diverse presentazioni di essi. Il problema che risolve è l'accoppiamento rigido fra dati e presentazione. Se abbiamo un oggetto Dati e più viste (tabella, grafico), senza Observer dovremmo chiamare a mano `tabella.aggiorna()` e `grafico.aggiorna()`, ogni volta che Dati cambia. Questo significa che Dati conosce esplicitamente tutte le sue viste, e ogni volta che ne viene aggiunta o tolta una, bisogna modificare la classe Dati stessa, e le classi perdono la possibilità di essere riusate in modo indipendente.
- L'idea è che il Subject, ovvero l'oggetto osservato, non debba sapere nulla di chi siano i suoi Observer. Li tratta tutti genericamente come oggetti che implementano un'interfaccia comune. Tramite il meccanismo della registrazione.

Observer in Java

- Il problema affrontato da questo D.P. è così comune che la sua soluzione è fornita nella libreria `java.util`, con i tipi **Observable** e **Observer**.
- Observable svolge il ruolo di Subject, quindi tiene traccia di tutti gli oggetti ConcreteObserver che vogliono essere informati di un cambiamento. Observable notifica il cambiamento di stato quando il metodo `notifyObservers()` viene chiamato.
- Perché notifyObservers() e non notify()? Perché notify() è già un metodo di `java.lang.Object`, la classe base di tutto. Esiste per **garantire sincronizzazione tra thread**.
- La classe Observable ha una variabile (**flag**) che indica se lo stato è cambiato dalla precedente notifica, ed è impostato dal metodo `setChanged()`. La chiamata

a setChanged() è da effettuare all'interno dei metodi della classe ConcreteSubject secondo la logica desiderata. Notiamo che se chiamiamo `notifyObservers()` alla fine del metodo senza controllare se lo stato è davvero cambiato, notifichiamo gli Observer per niente, e quindi `setChanged()` risolve questo problema. `notifyObservers()` non fa nulla se il flag non è impostato.

- Observer è una interfaccia che ha solo il metodo `update()` e può avere un argomento che indica quale oggetto ha causato l'aggiornamento.
- Quando il ConcreteSubject chiama notify(), questo chiama update() che **esegue sul thread del chiamante**, costringendo il chiamante ad aspettare l'esecuzione di update() di ciascun ConcreteObserver.
- Più recentemente, con i **Reactive Streams**, si è cercato di risolvere il problema del passaggio di un insieme di item da un **publisher** a un **subscriber**, **senza bloccare il publisher** e **senza inondare il subscriber**. Quindi i problemi che i Reactive Streams vogliono risolvere sono: publisher troppo veloce e publisher che si blocca.
- Con la versione 9 di Java, Observable e Observer sono disponibili per compatibilità con versioni precedenti, ma sconsigliati.
- Java 9 fornisce nella libreria `java.util.concurrent` le interfacce `Publisher<T>`, `Subscriber<T>`, `Subscription`, e la classe `SubmissionPublisher`. Quest'ultima implementa un Publisher dedicando un thread per mandare ciascun messaggio ai Subscriber.
- **Publisher** è l'astrazione per fornire item, definisce il metodo `subscribe()`, che prende in input `Subscriber<T>`
- **SubmissionPublisher** implementa Publisher, e invia item ai Subscriber in **maniera asincrona**. Quando sul SubmissionPublisher è chiamato il metodo `submit()`, esso esegue in un thread dedicato asincrono la chiamata al metodo `onNext()` dei Subscriber, e si blocca se il subscriber non può ricevere l'item.
- **Subscriber** è usato per ricevere item, e definisce i metodi `onComplete()` (chiamato quando non arrivano più item), `onError()`, `onNext()` (chiamato per ogni item ricevuto), `onSubscribe()`.
- **Subscription** definisce il collegamento fra Publisher e Subscriber, e i metodi `cancel()` e `request(n)` permettono al Subscriber di fermare l'invio di messaggi e

di richiedere l'invio dei prossimi n messaggi. Con cancel() il Subscriber dice "non voglio più messaggi", con request(n) dice "sono pronto a ricevere altri n item".

Sintesi:

1. Il problema risolto dall'Observer è così comune che Java offre l'implementazione pronta in `java.util`, con i tipi Observable e Observer. Notify è già occupato da `java.lang.Object`.
2. Meccanismo di `setChanged()` e `notifyObservers()` con un **flag** tenuto da Observable, variabile booleana che indica se lo stato è davvero cambiato dall'ultima notifica. Se chiamassimo `notifyObservers()` a prescindere, ogni chiaamta a un metodo del ConcreteSubject genererebbe una notifica anche quando lo stato non è effettivamente variato, sprecanto lavoro.
3. Da Java 9 Observable e Observer sono **deprecati e sconsigliati**, ma mantenuti per **retrocompatibilità**. Sono invece stati introdotte le interfacce `Publisher`, `Subscriber` e `Subscription` e la classe `SubmissionPublisher`, che implementano i **Reactive Streams**.
4. Motivi per questo cambio: Observable era una classe astratta e non un'interfaccia, e si consumava l'unico slot di ereditarietà disponibile in Java. Secondo motivo: la notifica era sincronae bloccante, bisogna aspettare l'esecuzione di update di ciascun Observer, rallentando l'esecuzione. Ultimo problema: publisher troppo veloce, subscriber che si intasa, o publisher che si blocca aspettando un subcriber lento. Con i metodi `cancel()` e `request(n)` viene dato al Subscriber il **controllo del flusso**. Permette di fermare l'invio dei messaggi e di richiedere l'invio dei prossimi n messaggi.
5. **Reactive Streams:** sono uno standard per gestire il flusso asincrono di dati fra un produttore e uno o più consumatori, risolvendo in modo esplicito il problema del produttore troppo veloce rispetto ai consumatori. In sostanza, vi è **l'aggiunta di un pool di thread condiviso**.

D.P. Model View Controller (MVC)

Intento: separare le **responsabilità** di un'applicazione interattiva in tre componenti distinti: **dati e logica** (Model), **presentazione** (View), **gestione dell'input**

(Controller), così che ognuno possa essere modificato, esteso e riusato indipendentemente dagli altri.

È considerato un **pattern architetturale** per le applicazioni interattive, che individua tre componenti: **Model** per funzionalità principali e dati; **View** per mostrare i dati; **Controller** per prendere gli input dell'utente.

Si chiama pattern architetturale perché non risolve un singolo problema di design come gli altri pattern, ma definisce come **organizzare l'intera struttura** di un'applicazione interattiva. L'idea è tenere separati tre ruoli che in un'applicazione naif finirebbero tutti nella stessa classe.

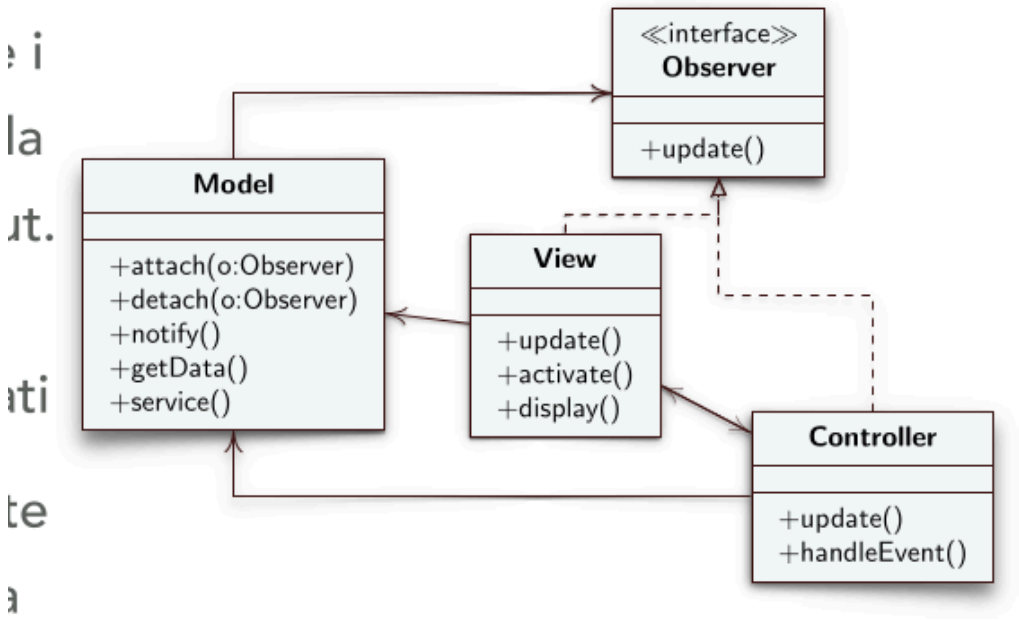
Motivazioni:

- Le interfacce utente possono cambiare, poiché funzionalità, dispositivi o piattaforme cambiano.
- Le stesse informazioni sono presentate in finestre differenti (per es. sotto forma di grafici diversi).
- Le visualizzazioni devono subito adeguarsi alle manipolazioni sui dati.
- I cambiamenti all'interfaccia utente dovrebbero essere facili.
- Il supporto ai diversi modi di visualizzare non dovrebbe avere a che fare con le funzionalità principali.

Soluzione:

- **Model:** incapsula le funzionalità principali e i dati dell'applicazione. È indipendente dalla rappresentazione degli output e dagli input. Registra View e Controller. Avvisa View e Controller registrati dei cambiamenti di dati. **Contiene i dati e la logica di business**, ovvero le operazioni che hanno senso nel dominio del problema. Tuttavia, **non sa nulla di come i dati vengono mostrati**, né di come l'utente interagisce.
- **View:** mostra i dati all'utente. Generalmente ci sono tante View, e ogni View è associata a un Controller. View inizializza il proprio Controller, e mostra i dati che legge da Model. Si occupa **solo di mostrare i dati** all'utente. Nel diagramma, View implementa l'interfaccia Observer, infatti **è un ConcreteObserver**.

- **Controller:** riceve gli input dell'utente (da mouse e tastiera) sotto forma di eventi. Traduce gli eventi in richieste di servizio per Model o avvisa View. Si occupa di **gestire l'input dell'utente**. Traduce gli eventi fisici in operazioni. Anche Controller **è un ConcreteObserver**.



Collegamento con Observer:

- **Model** ha operazioni `attach()`, `detach()`, `notify()`. Rappresenta il **Subject**.
- **View** e **Controller** rappresentano i **ConcreteObserver**.
- Model non sa che esistono View e Controller come tipi concreti: sa solo dell'esistenza di Observer, e li conosce come tali. **Disaccoppiamento**.
- **Dipendenza reciproca ma asimettrica** tra View e Controller: View inizializza il proprio Controller, e quindi crea e tiene un riferimento al suo Controller perché l'utente fa qualcosa sull'interfaccia (click, tasto). È la View che riceve fisicamente l'evento. Non sa come gestirlo, e quindi chiama `handleEvent()`.
  Il Controller, invece, tiene un riferimento a View per poter chiamare `activate()` direttamente su di essa, senza passare per il Model. Serve per comandi UI che non riguardano i dati: mostrare un dialogo di errore, evidenziare un campo, aprire un pannello. Queste sono **operazioni di interfaccia pura** che non hanno

senso nel Model. View e Controller **sono una coppia**: ogni View ha esattamente un Controller associato, e viceversa. Si occupano dello stesso pezzo di interfaccia: uno dalla parte dell'input e uno dalla parte dell'output.

- Ogni **pagina** è tipicamente una View. Non esistono View aperte simultaneamente che si aggiornano: ogni richiesta produce una nuova View da zero. Una View è un **modo distinto di rappresentare i dati**: tabella, grafico, form, pagina. Questa relazione con Observer **ha senso solo se View ha bisogno di dati dinamici**. Se non è così, e la View mostra un contenuto statico (come pagina di login, schermata "about", pagina di errore, etc), allora non ha nessun motivo di fare `attach()` sul Model, non chiama `getData()` e il Model non la notifica. Il pezzo Observer della relazione non si attiva. Tuttavia, è ancora una View nel senso MVC, dato che ancora mostra qualcosa all'utente. Deve semplicemente **non registrarsi come Observer sul Model**. Tuttavia, comunque useremo l'MVC per **organizzazione generale**, dato che questa rappresenta l'architettura dell'intera applicazione, e non qualcosa che accendiamo o spegniamo per la singola pagina. È la struttura dentro cui tutto vive, anche le parti semplici come quelle statiche.

Flusso completo:

- L'utente clicca qualcosa sulla View.
- View delega al suo Controller tramite `handleEvent()`.
- Il Controller chiama `service()` sul Model per richiedere un'operazione.
- Model aggiorna il suo stato interno, e poi chiama `notify()`.
- A questo punto, `notify()` chiama `update()` su tutti gli Observer registrati (View e Controller).
- View in `update()` chiama `getData()` sul Model per leggere i nuovi dati, poi `display()` per disegnare.

Perché Controller implementa Observer?

- Il Controller non serve solo per passare input al Model. Può anche dover **reagire ai cambiamenti del Model** per aggiustare il suo comportamento.
- **Esempi:** Model entra in uno stato "sola lettura", e il Controller deve sapere che non può più accettare certi input. Oppure Model carica dei dati, e il Controller

abilita certi pulsanti che prima erano disabilitati.

- Se il Controller non implementasse Observer, dovrebbe interrogare il Model attivamente o ricevere notifiche in modo diverso.

La separazione di Model, View e Controller serve a rendere le tre parti **modificabili e riusabili indipendentemente**. MVC è Observer applicato su larga scala a un'intera applicazione.

Esempi reali di MVC:

- Nei **sistemi distribuiti e web**, non c'è l'MVC classico (GUI desktop), dove tutti e tre i componenti vivono sulla stessa macchina. Nel web MVC la separazione diventa anche fisica: Model e Controller risiedono **lato server**, e la View viene renderizzata nel **browser del client**. La comunicazione avviene via HTTP: il Controller riceve richieste HTTP, interroga il Model e restituisce una View (pagina HTML) al browser. **Spring Boot** è il framework Java più usato per realizzare questa architettura.
- **Model svolge il ruolo di Subject e ConcreteSubject**: è Subject perché gestisce attach, detach, notify, ed è ConcreteSubject perché tiene anche i dati e la logica al suo interno.

D.P. State

Design Pattern **comportamentale**. Ricordiamo che i pattern comportamentali riguardano come gli oggetti collaborano e distribuiscono le responsabilità.

Intento: permettere a un oggetto di alterare il suo comportamento quando il suo stato interno cambia. Far sembrare che l'oggetto abbia cambiato la sua classe.

Con questo Design Pattern, dal punto di vista del client, chiamare lo stesso metodo sullo stesso oggetto produce comportamenti radicalmente diversi a seconda dello stato, come se stesse interagendo con oggetti di classi diverse. È il **polimorfismo** che crea questa illusione.

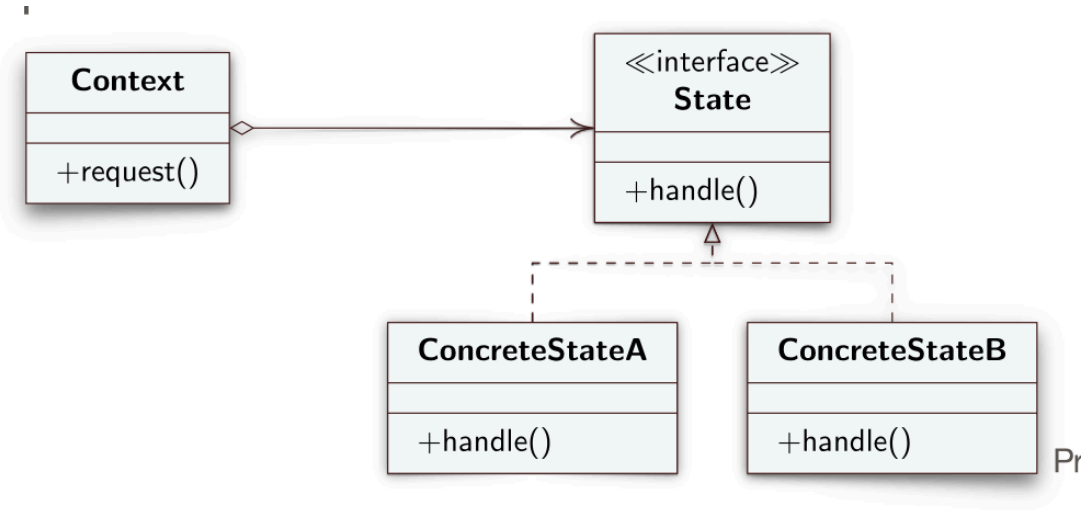
Problema:

- Il comportamento di un oggetto dipende dal suo stato e il **comportamento deve cambiare a run-time** in base al suo stato.

- Le operazioni da svolgere hanno alcuni **grandi rami condizionali** che dipendono dallo stato. Il problema non è tanto la quantità, ma è che questi rami condizionali (tipicamente `switch` o catene di `if/else`) sono **interni a ogni singola operazione della classe**, rendendo ogni metodo verboso e difficile da leggere.
- Lo stato è spesso rappresentato dal valore di una o più **variabili enumerative** costanti.
- Spesso varie operazioni contengono la **stessa struttura condizionale**. Questo significa, come detto prima, che ogni metodo ha uno `switch` sullo stato e ognuno di questi ha la stessa struttura, duplicata su più metodi. Viola il **DRY** in modo sistematico.
- La **manutenibilità non esiste**: aggiungere un nuovo stato significa aprire ogni singola operazione e modificare ogni singolo ramo condizionale. Il codice viola l'**Open/Closed Principle**.

Soluzione:

- Inserire ogni ramo condizionale in una classe separata. Ogni `case` del vecchio `switch(state)` diventa una classe concreta dedicata. Il comportamento specifico di quella classe è **incapsulato** in quella classe, non più sparso in ogni metodo del Context.
- **Context**: definisce l'interfaccia che interessa ai client, e mantiene un'istanza di una classe ConcreteState che definisce lo stato corrente. Context ha un campo privato `currentState` di tipo `State`, e il metodo `request()` non fa altro che delegare a `currentState.handle()`. Il client chiama sempre e solo il Context, senza sapere nulla degli stati interni.
- **State:** definisce un'interfaccia che incapsula il comportamento associato ad un particolare stato del Context. O meglio, State non incapsula il comportamento, lo dichiara tramite un'interfaccia. È poi ogni ConcreteState che lo incapsula davvero nella propria implementazione.
- **ConcreteState:** sono le sottoclassi che implementano ciascuna il comportamento associato ad uno stato del Context. **Ogni ConcreteState sa fare una cosa sola**, ovvero il comportamento corretto per quello stato specifico. Nessun `if` o `switch` al suo interno.



Descrizione del diagramma:

- Da Context parte una frecca con **rombo vuoto** verso State: è una **aggregazione**. Context ha un riferimento a un oggetto State (il suo stato corrente), ma non ne è il proprietario esclusivo. Gli stati potrebbero teoricamente essere condivisi o preesistenti.
- **Perché non è una semplice dipendenza?** Perché qui Context mantiene un **riferimento strutturale** a State, e State è salvato come **campo dell'oggetto**, e quindi persiste tra una chiamata e l'altra. L'uso di State all'interno del Context non è temporaneo, ma è intrinseco a Context. Il rombo ci dà questa informazione: State è una parte del Context, ma State può esistere anche senza Context.

Collaborazioni:

- Il Context passa le richieste dipendenti da un certo stato all'oggetto ConcreteState corrente. Meccanismo centrale del pattern: **delega**. Quando il client chiama `context.request()`, il Context internamente chiama `currentState.handle()`. Il client non sa quale stato è attivo, e non gli interessa.
- **Un Context può passare sé stesso come argomento all'oggetto ConcreteState** per farlo accedere al contesto se necessario. Il ConcreteState, infatti, potrebbe aver bisogno di: leggere dati dal Context (come un contatore o un valore) e di cambiare lo stato del Context (es. `context.setState(new`

`ConcreteState B))`. Per questo `handle()` spesso accetta il Context come parametro. Senza di questo il ConcreteState sarebbe cieco rispetto a chi lo ha invocato.

- Il Context è l'interfaccia per le classi Client. Il Client, infatti, non ha mai un riferimento diretto a ConcreteStateA o a ConcreteStateB. **Incapsulamento**: il client ignora completamente l'esistenza degli stati concreti.
- Il Context o i ConcreteState decidono quale stato è il successivo e in quali circostanze. Questo è un punto di design non vincolato dal pattern. Sono due gli approcci validi:
  1. **Context gestisce le transizioni**: logica centralizzata, più facile da leggere globalmente.
  2. **ConcreteState gestisce le transizioni:** ogni stato sa da solo cosa viene dopo, più flessibile ma la logica è distribuita fra le classi. Qui **la transizione dipende dallo stato corrente**. È `LockedState`, per esempio, che sa che un gettone lo porta ad `Unlocked`. Il Context non lo sa senza prima ricreare qual è lo stato attuale, e ciò gli farebbe ricreare quella catena di `if/else` che volevamo eliminare. In questo modo, ogni stato conosce il proprio comportamento e le proprie transizioni.

Conseguenze:

- Il comportamento associato a uno stato è localizzato in una sola classe (**ConcreteState**) e si partiziona il comportamento di stati differenti. Per tale motivo, si possono aggiungere nuovi stati e transizioni facilmente, creando nuove sottoclassi. Incapsulare le azioni di uno stato in una classe impone una struttura e rende più chiaro lo scopo del codice.
- La logica che gestisce il cambiamento di stato è separata dai vari comportamenti ed è in una sola classe (**Context**), anziché (con istruzioni if o switch) sulla classe che implementa i comportamenti. Tale separazione aiuta a evitare stati inconsistenti, poiché i cambiamenti di stato vengono decisi da una sola classe e non da tante.
- Quando le transizioni sono gestite dai ConcreteState, e quindi l'interfaccia usata dai ConcreteState permette di ritornare il riferimento a un nuovo state, si parla di **interfaccia fluent**.

- Il numero di classi totale è maggiore, ma le classi sono più semplici.

Tecnica di Refactoring:

- Al Design Pattern State è spesso associata una tecnica di refactoring, ovvero il **Replace Conditional with Polymorphism** (sostituisci i rami condizionali con il polimorfismo). Quindi, nel nostro caso specifico, si tratta di **Replace Type Code with State**. Il type code è un campo usato per rappresentare il tipo o lo stato di un oggetto, sul quale poi si ramifica il comportamento dell'oggetto. Es. type = 1 per indicare stato disponibile, e così via. Porta a diversi problemi, come a **comportamento sparso** (la logica per ciascun valore sta tutta dentro lo switch) e ha **scarsa estensibilità**, in quanto viola il principio **Open/Closed**.
- La presenza di **switch** è un **sintomo che suggerisce di usare il polimorfismo**.

Sintesi:

- Pattern Comportamentale il cui intento è quello di permettere a un oggetto di alterare il proprio comportamento quando cambia il suo stato interno, facendo sembrare che l'oggetto abbia cambiato classe dal punto di vista del client.
- Il problema che risolve è quello dei grandi rami condizionali interni a ogni operazione. Lo state è spesso rappresentato da una variabile enumerativa (**type code**), e ogni metodo della classe contiene uno switch o una catena di if/else su quella variabile per decidere cosa fare. Il problema è che questa struttura logica condizionale si ripete su ogni metodo, violando il **DRY** e violando il principio **Open/Closed**, non permettendo la manutenibilità.
- L'idea è di prendere ogni case dello switch è trasformarlo in una classe concreta dedicata, incapsulando in essa il comportamento specifico, invece di lasciarlo sparso per ogni metodo.
- **Non bisogna passare un typecode** per passare da uno stato all'altro, ed è anche questo che il pattern risolve. La transizione non è più data da un confronto su un valore, ma una sostituzione di riferimento. Il "quale stato sono" non è più un dato da leggere e confrontare, ma è incapsulato nell'identità stessa dell'oggetto ConcreteState a cui il Context punta in quel momento. Poi le **transizioni** o sono gestite dal Context stesso, e quindi è lui a "settare" il nuovo ConcreteState, oppure sono i ConcreteState a deciderlo. Per esempio, potremmo chiamare un `handleRequest()` su un ConcreteState, e questo potrebbe

ritornarci un nuovo oggetto ConcreteState (un altro ancora), in quanto si è occupato lui stesso di modificare lo stato interno del Context. O il nuovo oggetto viene ritornato (e di conseguenza assegnato alla variabile che riguarda lo State nel Context), oppure il Context ha passato un riferimento a sè stesso, e il ConcreteState ha chiamato il metodo `setState()` del Context.

D.P. Chain Of Responsibility

Design Pattern **comportamentale**.

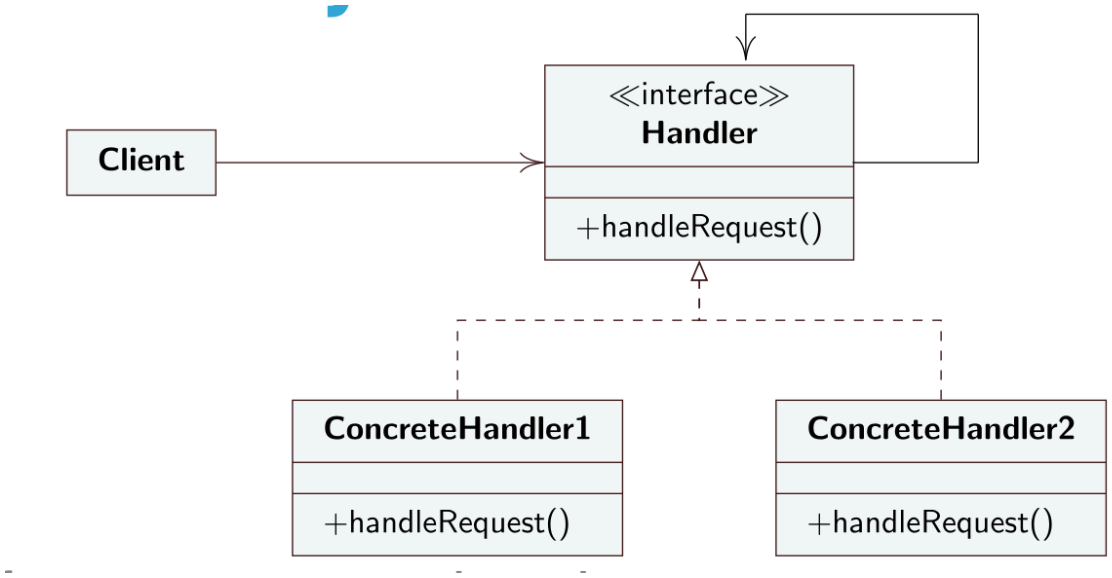
Intento: evitare di accoppiare il mandante di una richiesta col suo ricevente, dando così a più di un oggetto la possibilità di gestire la richiesta. Concatena gli oggetti riceventi e passa la richiesta lungo la catena fino a che un oggetto la gestisce.

Motivazioni:

- Consideriamo un'interfaccia utente in cui l'utente può chiedere aiuto su parti dell'interfaccia. Per es. un bottone può fornire informazioni di aiuto. Se non esiste un'informazione specifica allora si dovrebbe fornire il messaggio di aiuto del contesto più vicino, es. la finestra. La catena **non è arbitraria**, ma **segue la gerarchia di contenimento del widget**. Il bottone è contenuto in un dialog, il dialog in una finestra applicativa, e così via. La catena rispecchia questa struttura compositiva.
- **Problema:** l'oggetto che fornirà il messaggio d'aiuto non è conosciuto dall'oggetto che effettua la richiesta iniziale. Bisogna **disaccoppiare** mandante e ricevente.

Soluzione:

- **Handler:** definisce l'interfaccia per gestire le richieste. `Handler` ha una **freccia su sé stesso**. Rappresenta il **riferimento al successore**. Ogni Handler conosce un altro Handler, il prossimo nella catena.
- **ConcreteHandler:** gestisce la richiesta di cui è responsabile, può accedere al suo successore, e inoltra la richiesta se non può gestirla
- **Client:** inizia effettuando la richiesta a un ConcreteHandler.
- La richiesta si propaga lungo la catena finché un ConcreteHandler la gestisce.



Conseguenze:

- Riduce l'accoppiamento: chi effettua una richiesta non conosce il ricevente.
- Un oggetto della catena non deve conoscere la struttura della catena, ma **deve conoscere solo il suo successore immediato**. Nota che l'oggetto handler, quindi, non ha un riferimento a una lista di oggetti, ma semplicemente all'handler successivo.
- Si aggiunge **flessibilità** nella distribuzione di responsabilità agli oggetti. Si può cambiare o aggiungere responsabilità nella gestione di una richiesta **cambiando la catena a runtime**. Si può inserire un nuovo handler, rimuoverne uno, o riordinare la catena senza toccare né il Client né gli altri handler.
- **Non vi è garanzia che una richiesta sia gestita**, poiché **non c'è un ricevente esplicito**. La richiesta potrebbe arrivare alla fine della catena senza essere gestita.
- **Svantaggio: overhead di attraversamento**. Se la catena è lunga e la richiesta arriva in fondo, tutti gli handler vengono interrogati inutilmente. In scenari con molte richieste e catene lunghe questo può avere un impatto sulle performance.
- **Svantaggio: difficoltà di Debugging**. Non è immediato capire chi ha gestito una richiesta, o perché non è stata gestita. Il flusso di controllo è implicito e distribuito nella catena.

Implementazione:

- Handler o ConcreteHandler possono definire i link per avere il successore della catena. Quindi ci sono due casi:
  1. **Handler** contiene l'attributo `successor` e fornisce un'implementazione di default di `handleRequest()` che inoltra automaticamente.
     I ConcreteHandler ereditano questo comportamento e lo sovrascrivono solo dove gestiscono qualcosa. Questo è l'approccio adottato nel diagramma precedente.
  2. **ConcreteHandler:** ogni ConcreteHandler gestisce autonomamente il proprio riferimento al successore. **Più controllo, ma codice duplicato**.
- Una gerarchia già esistente può essere usata, anziché ridefinire i link (il design pattern **Composite** è usato insieme a Chain).
- Il Design Pattern deve sapere **quale tipo di richiesta** sta circolando nella catena, così ogni handler può decidere "è per me o la passo avanti?". La domanda implementativa è: **come rappresentiamo il tipo di richiesta nel codice?** Ci sono due approcci possibili.
  1. **Approccio 1: Un metodo per tipo di richiesta.**
     Ogni tipo di richiesta diventa un metodo separato nell'interfaccia. Es. avremo il metodo `handleHelp()`, il metodo `handlePrint()`, e così via. Il Client chiama `handler.handleHelp()`, e il tipo di richiesta è già **codificato nella firma del metodo**. Non serve controllare nulla a runtime. Il compilatore sa già che stiamo chiedendo "help".
     Problema: quando vogliamo aggiungere un nuovo tipo di richiesta, dobbiamo **aggiungere un nuovo metodo all'interfaccia**. E tutti i ConcreteHandler esistenti **devono implementarlo** (anche solo per far passare la richiesta al successore). Modificare l'interfaccia **significa ricompilare tutto**. Questo approccio è **type-safe** in quanto è il compilatore che garantisce che stiamo usando richieste valide, e non un controllo a runtime.
  2. **Approccio 2: Un solo metodo con un codice.**
     L'interfaccia ha un solo metodo generico, e il tipo di richiesta viene passato come parametro. Il client chiama `handler.handleRequest(1)`, dove, per esempio, il codice 1 è associato alla richiesta "help". Ogni ConcreteHandler

fa un controllo a runtime, controllando il valore del codice.
Differenze con l'approccio 1: l'interfaccia resta sempre la stessa, ovvero `void handleRequest(int)`. Per aggiungere un nuovo tipo di richiesta basta definire un nuovo codice, e far sì che un handler lo riconosca. **Zero modifiche all'interfaccia**. La differenza sta in **cosa bisogna modificare quando il sistema cresce**. Nel caso 1, bisognava modificare l'interfaccia e di conseguenza tutti i ConcreteHandler, per poi ricompilare tutto. Nel caso 2, invece, ci basta aggiungere un valore intero e aggiungere nel ConcreteHandler corrispondente (o corrispondenti) il codice per gestirlo. Nessuna modifica all'interfaccia. **Svantaggio:** se passiamo un codice che nessun handler conosce, la richiesta attraversa tutta la catena silenziosamente, e non ci viene dato nessun errore e nessun avviso; poiché non è type-safe nessuno ci dirà che quella richiesta non la gestisce nessuno, dobbiamo capirlo noi dal runtime. **Scelte di design che impattano l'estensibilità**.

3. Quindi, nel caso dell'**Approccio 2**, con il codice possiamo verificare facilmente se la richiesta è destinata a quel ConcreteHandler o no: ci basta vedere se è presente il ramo condizionale con quel codice. Con l'**Approccio 1**, invece, per capire se la richiesta è destinata a quel ConcreteHandler bisogna semplicemente entrare nel metodo della richiesta (il singolo metodo), e se quel metodo si occupa della richiesta bene, sennò la richiesta verrà passata al prossimo ConcreteHandler.

- Ciascun tipo di richiesta può corrispondere a un'operazione, in questo caso il set di richieste è definito dall'Handler. Ogni tipo di richiesta diventa un **metodo distinto** nell'interfaccia Handler.

  1. **Vantaggio:** type-safe, chiaro, ogni richiesta ha la sua firma.
  2. **Svantaggio:** il set di richieste è **fisso a compile-time**. Se vogliamo aggiungere un nuovo tipo di richiesta dobbiamo modificare l'interfaccia `Handler`, e di conseguenza tutti i ConcreteHandler esistenti. Non rispetta l'**Open/Closed Principle**.

- Ciascun tipo di richiesta corrisponde a un codice e solo un'operazione è definita dall'Handler. Il set di richieste non è fissato: richiedenti e riceventi prendono accordi sulla codifica delle richieste. Si hanno più controlli a runtime. Si avrà così un solo metodo generico in Handler, con un parametro che

codifica il tipo di richiesta. Ogni handler deve **controllare a runtime** il codice ricevuto per decidere se può gestirlo.

1. **Vantaggio:** il set di richieste **non è fisso**. Si possono aggiungere nuovi codici senza toccare l'interfaccia Handler.
2. **Svantaggio:** meno sicurezza compile-time, più logica condizionale nei ConcreteHandler, errori nel codice richiesta rilevabili solo a runtime.

Sintesi:

- Pattern Comportamentale. Il suo intento è quello di evitare di accoppiare il mandante di una richiesta al suo ricevente, dando a più di un oggetto la possibilità di gestirla. Gli oggetti riceventi vengono concatenati, e la richiesta viagga lungo la catena finché uno di essi la gestisce.
- Il problema che risolve è quello del mandante che non sa, e **non deve sapere**, chi alla fine soddisferà la sua richiesta. **Esempio classico:** sistema di help in un'interfaccia grafica; un bottone per chiedere aiuto, e se non ha un messaggio specifico, la richiesta dovrebbe risalire al contesto più ampio. Il dialog che lo contiene, la finestra applicativa, e così via. La catena non è arbitraria, ma rispecchia la gerarchia di contenimento già esistente fra i widget. Si potrebbe usare insieme al **Composite**.
- **Conseguenze notevoli:** riduce l'accoppiamento, infatti chi richiede non conosce chi gestirà, e ogni oggetto della catena deve conoscere solo il proprio successo immediato, e non l'intera struttura, e si guadagna flessibilità potendo **riordinare**, aggiungere o togliere handler a runtime senza toccare il Client o gli altri elementi della catena (tranne magari l'immediato predecessore).
- **Svantaggi:** non c'è garanzia che la richiesta venga gestita, c'è un overhead di attraversamento se la catena è lunga e la richiesta arriva in fondo, e c'è seria difficoltà di debugging.
- **Approcci implementativi, Modo 1:** un metodo per richiesta, e quindi Handler dichiara un metodo distinto per ciascun tipo. È **type-safe**, in quanto è il compilatore a garantire che la richiesta sia valida, non serve nessun cointrollo a runtime. **Svantaggio:** set di richieste fisso a compile-time, aggiungere un nuovo tipo significa modificare Handler e tutti i ConcreteHandler, violando l'**Open/Closed Principle**, e dovendo ricompilare tutto.

- **Approcci implementativi, Modo 2:** si ha solo un metodo generico con un codice come parametro, es. `handleRequest(int code)`, e l'interfaccia non cambia mai. Per aggiungere un nuovo codice basta che qualche ConcreteHandler lo riconoscal al suo interno, con un **controllo a runtime**.
  Vantaggio: zero modifiche all'interfaccia quando il sistema cresce.
  Svantaggio: si perde la sicurezza **compile-time**, se passiamo infatti un codice che nessun handler conosce, la richiesta non viene gestita da nessuno, attraversando silenziosamente tutta la catena. Lo capiremo solo osservando il comportamento a runtime, **nessun errore esplicito**.

D.P. Command

Intento: incapsulare una richiesta in un oggetto, permettendo quindi di parametrizzare i client con differenti richieste, code o log di richieste, e supportare l'annullamento di operazioni.

Parametrizzata il client: si intende **iniettare** il comportamento come se fosse un parametro.

Questo significa che possiamo passare a un oggetto Invoker comandi diversi senza che lui sappia cosa fa ciascuno, esattamente come si passa un argomento a una funzione. Per quanto riguarda l'annullamento di operazioni, questo è possibile in quanto sarà possibile implementare un metodo `undo()`.

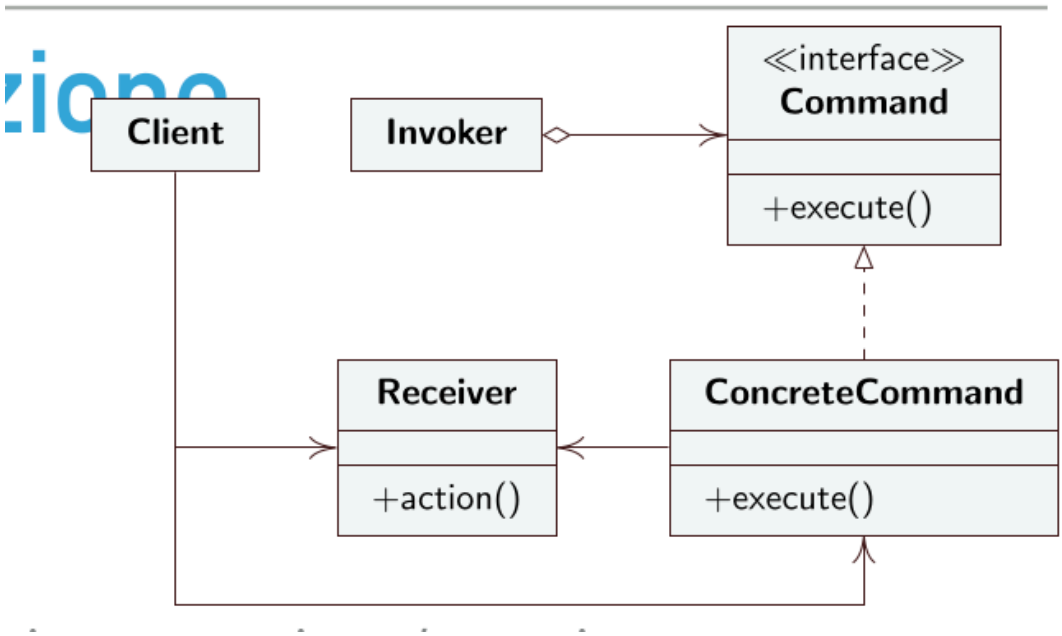
Motivazioni:

- Qualche volta può essere necessario mandare una richiesta a un oggetto senza conoscere alcunché dell'operazione richiesta o del ricevente della richiesta.
  Chi emette la richiesta non deve sapere chi la esegue né come.
- La **richiesta viene trasformata in oggetto** e quest'oggetto può essere immagazzinato o passato come altri oggetti. Passare la chiamata in un oggetto ci dà tutti i vantaggi della programmazione a oggetti su quel comando: passarlo, salvarlo, comporlo.
- La chiave di questo pattern è una classe astratta Command che dichiara un'interfaccia per eseguire operazioni. Le sottoclassi specificano la coppia ricevente e azione, immagazzinando il ricevente come attributo e

implementando il metodo che effettua, o per meglio dire **delega**, le richieste. Il ricevente conosce ciò che è necessario per servire la richiesta.

Soluzione:

- **Command** definisce un'interfaccia per eseguire un'operazione. Dichiara solo il contratto `execute()`. È una semplice interfaccia che non contiene logica.
- **ConcreteCommand** definisce il collegamento fra un oggetto Receiver e un'azione. Classe concreta che implementa l'operazione `execute()` chiamando le corrispondenti operazioni sul Receiver. Si occupa quindi di **delegare** il lavoro al Receiver.
- **Client** crea un oggetto ConcreteCommand e imposta il suo Receiver. Per questo motivo conosce sia il ConcreteCommand che il Receiver: li crea e li collega. È l'unico partecipante che conosce tutto, infatti è lui che sa quale ConcreteCommand usare e quale Receiver assegnargli.
- **Invoker** chiede al comando di effettuare la richiesta. Tiene infatti un riferimento a Command. Qui si parla di **aggregazione** in quanto Invoker contiene un riferimento a Command, ma non ne è il proprietario esclusivo. **Disaccoppiamento:** l'Invoker punta a Command, non a ConcreteCommand, ed è questo il cuore del disaccoppiamento, dato che non sa mai con quale comando concreto sta lavorando. Non sa nulla di Receiver né di ConcreteCommand. Tiene solo un riferimento all'interfaccia Command, e al momento opportuno chiama `execute()`. Potrebbe essere un bottone della GUI, uno scheduler, un gestore di input, qualsiasi cosa che scatena l'azione.
- **Receiver** conosce come effettuare le operazioni associate alla richiesta. Qualunque classe può essere un Receiver. Classe con un metodo `action()` che rappresenta la logica applicativa reale.



Collaborazioni:

- Il client crea un oggetto ConcreteCommand e specifica il suo oggetto Receiver. Fase di **configurazione**, avviene una volta sola (o all'inizio di ogni operazione). Il client è l'unico che conosce entrambe le parti e decide l'abbinamento.
- Un oggetto Invoker tiene l'oggetto ConcreteCommand. L'Invoker riceve il comando dal Client e lo **memorizza** come riferimento all'interfaccia Command. Non mostriamo la freccia `Client --> Invoker` in quanto il diagramma delle classi mostra solo le **relazioni strutturali permanenti** tra le classi, e non ogni interazione che avviene a runtime. Invoker è una **configurazione una tantum**, infatti il Client poi non mantiene nessuna associazione duratura con lui. L'associazione sarebbe invece visibile in un diagramma di sequenza.
- L'oggetto Invoker effettua una richiesta chiamando `execute()` sull'oggetto Command. Quando i comandi sono **reversibili**, il ConcreteCommand conserva lo stato necessario per cancellare il comando e ritornare a prima della chiamata a `execute()`. Questo stato viene aggiunto a uno **stack** nell'Invoker. Se si vuole annullare, l'Invoker chiama `undo()` sull'ultima chiamata dello stack, che usa lo stato salvato per ripristinare la situazione. Se il comando non è reversibile, allora viene semplicemente chiamato `execute()` e basta.

- L’oggetto ConcreteCommand chiama le operazioni sul suo Receiver per portare a termine la richiesta.

Cosa intendiamo con operazioni?

- Supponiamo di usare un **editor di testo**. Ogni cosa che facciamo è un comando, quindi per esempio scrivere “ciao”, cancellare una parola, metterne una in grassetto e così via.

Conseguenze:

- Il pattern Command **disaccoppia** l’oggetto che invoca l’operazione da quello che conosce come effettuarla.
- I comandi diventano oggetti e possono essere manipolati ed estesi come altri oggetti. Siccome i comandi, es. `DeleteCommand`, sono oggetti, possiamo passarli come argomento, metterli in una lista, salvarli su disco, copiarli, etc.
- Si possono assemblare comandi in un comando composto (**MacroCommand**). In generale, i comandi composti sono un’istanza del design pattern Composite.
- È facile aggiungere nuovi comandi, poiché non bisogna cambiare classi esistenti.

Implementazione:

- Un Command può avere un’ampia gamma di abilità. Potrebbe solo definire il legame con un ricevente e l’azione da eseguire. Oppure, implementare tutto facendo così a meno del ricevente.
  Può quindi **contenere solo il riferimento al Receiver e chiamarlo**, oppure **contenere tutta la logica dentro di sé, senza Receiver**. Usiamo il secondo approccio quando la logica è talmente semplice e isolata che creare un Receiver separato sarebbe eccessivo.
- I **dati necessari all’operazione** vengono passati al ConcreteCommand quando viene creato, non quando viene eseguito. Quando l’Invoker chiama `execute()` non passa nulla, in quanto i dati sono già dentro il comando.
- **Ogni operazione è un oggetto separato**. Avere un oggetto per ogni operazione è esattamente ciò che rende possibile undo/redo a più livelli. Se riusassimo lo stesso oggetto sovrascrivendo i dati, perderemmo la storia di tutto ciò che abbiamo fatto.

- I Command possono supportare **undo** se hanno la capacità di rendere reversibile la loro esecuzione. Per questo la classe ConcreteCommand potrebbe aver bisogno di conservare uno stato aggiuntivo. Il ricevente deve fornire le operazioni che permettono al command di far tornare il ricevente allo stato precedente. Il Receiver per ogni operazione **deve fornire l'operazione inversa**.
- Per un livello di undo, il command deve conservare l'attività eseguita, per più livelli di undo (e di **redo**), sarà necessario avere una lista di comandi eseguiti, quindi uno **stack**. Per il redo potremmo tenere un secondo stack.

Sintesi:

- Il Command è un pattern comportamentale (in quanto incapsula una richiesta, e gestire richieste è proprio il tipo di cooperazione fra oggetti di cui si occupano i pattern comportamentali). Il suo intento è incapsulare una richiesta dentro un oggetto, in modo da poter parametrizzare i client con richieste diverse, accodarle, loggarle, e supportare l'annullamento di tali operazioni.
- **Parametrizzare il client:** iniettare un comportamento come se fosse un dato qualsiasi. Possiamo passare a un oggetto Invoker comandi diversi senza che lui sappia cosa fanno realmente, esattamente come passiamo un argomento a una funzione.
- Il problema che risolve è quello di mandare una richiesta a un oggetto senza dover conoscere né l'operazione richiesta né chi effettivamente la eseguirà. Chi emette la richiesta non deve sapere chi la esegue né come. Trasformando la richiesta in un oggetto possiamo sfruttare i vari vantaggi della programmazione a oggetti, es. passarla, salvare, e **comporla** con altre richieste.
- Il Command può limitarsi a contenere un riferimento al Receiver e delegargli tutto, oppure, quando la logica è semplice e isolata, può contenere tutta la logica al proprio interno, facendo a meno del Receiver. Quando Invoker chiama `execute()` non passa nulla, perché tutto ciò che serve è già dentro l'oggetto comando.
- **Conseguenze:** il pattern disaccoppia chi invoca l'operazione e chi sa come eseguirla. I comandi sono oggetti veri e propri, e possono essere passati,

salvati, copiati, messi in lista, composti in **MacroCommand**. Aggiungere nuovi comandi è semplice, perché non richiede modifiche alle classi esistenti.

- **Meccanismo di undo/redo:** avere un oggetto separato per ogni operazioni è ciò che lo rende possibile. Se riusassimo lo stesso oggetto sovrascrivendo i dati a ogni esecuzione, perderemmo la cronologia di tutto ciò che è stato fatto. Per supportare undo, il Receiver deve fornire l'operazione inversa per ciascuna azione, e il ConcreteCommand deve conservare lo stato necessario per tornare indietro. Quando i comandi sono reversibili, questo stato viene aggiunto a uno stack mantenuto nell'Invoker. Se si vuole annullare, l'Invoker chiama `undo()` sull'ultimo elemento dello stack. Se si vuole supportare la `redo()` servirà un altro stack.
- **Esempio classico:** quello di un Editor di Testo, dove i comandi sono degli oggetti che hanno un riferimento all'Editor e contengono del testo passato dal costruttore. Per cancellare quel testo si usa undo(), e per farla basta cancellare le lettere finali del testo presente nell'editor, date dalla lunghezza del testo scritto dall'ultimo comando.

D.P. Mediator

Intento: definire un oggetto che **incapsula come un gruppo di oggetti interagisce**. Il Mediator promuove il lasco accoppiamento fra oggetti poiché evita che essi interagiscano direttamente, e permette di modificare le loro interazioni indipendentemente da essi.

Caratteristica principale: stiamo incapsulando tutte le interazioni fra gli oggetti, non un singolo comportamento o una singola relazione. Gli oggetti che collaborano smettono di conoscersi a vicenda e conoscono solo l'interfaccia del Mediator.

Motivazioni:

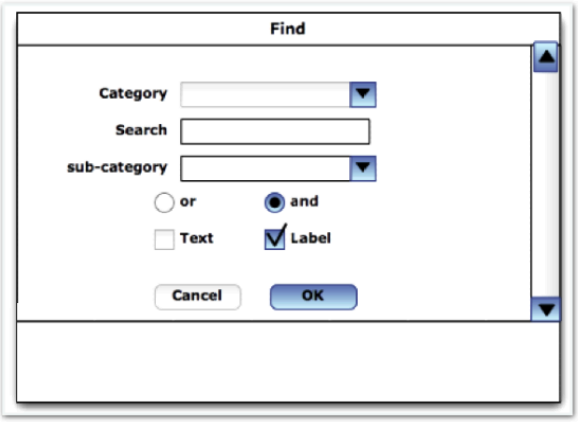
- La **distribuzione di responsabilità** fra vari oggetti può risultare in molte connessioni fra oggetti, e nel caso peggiore **un oggetto conosce tutti gli altri**.
- Molte connessioni rendono un oggetto dipendente da altri e l'intero sistema si comporta come se fosse **monolitico**.
  Inoltre potrebbe essere **difficile cambiare il comportamento del sistema**

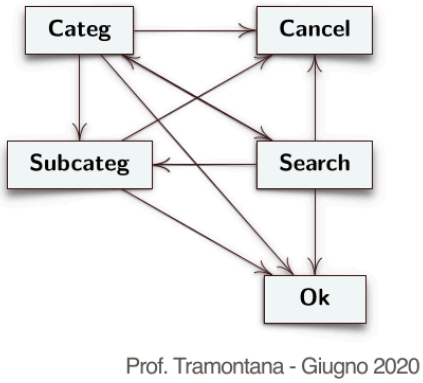
poiché il comportamento è distribuito fra oggetti. In questo modo riusare un singolo oggetto fuori contesto diventa quasi impossibile, perché per usarlo dovremmo anche portarci dietro tutta la rete di oggetti a cui è collegato.

- Si possono evitare questi problemi incapsulando il comportamento collettivo in un **oggetto mediatore separato**. Il mediatore serve da intermediario ed evita che gli oggetti dipendano fra loro. Il Mediator non elimina la dipendenza, la **ridireziona**. I Colleague dipendono ancora da qualcosa: da un'unica interfaccia stabile Mediator, e non da classi concrete degli altri Colleague.

Esempio:

- Supponiamo di sviluppare una finestra di ricerca. Ogni elemento visualizzato (testo, lista, bottone, etc) è controllato da una corrispondente classe.
- Ciascuna classe deve comunicare il suo stato alle altre per far aggiornare la visualizzazione.
- Ciascuna classe, senza un Mediator, chiamerà i metodi di tutte le altre classi, e quindi ciascuna classe è dipendente dalle altre. Per esempio: se cambiamo la Category, allora la lista delle Sub-Category disponibili deve aggiornarsi di conseguenze. Se il campo Search è vuoto, il bottone Ok probabilmente dovrà restare disabilitato perché non c'è nulla da cercare. Non è un caso che Ok sia il nodo con più freccette entranti: il suo stato enabled/disabled dipende, in pratica, dalla combinazione di tutti gli altri input.
- Se volessimo riusare la classe Search da qualche altra parte, non possiamo staccarla facilmente, in quanto porta con sé chiamate dirette a Category, SubCategory, Cancel e Ok.

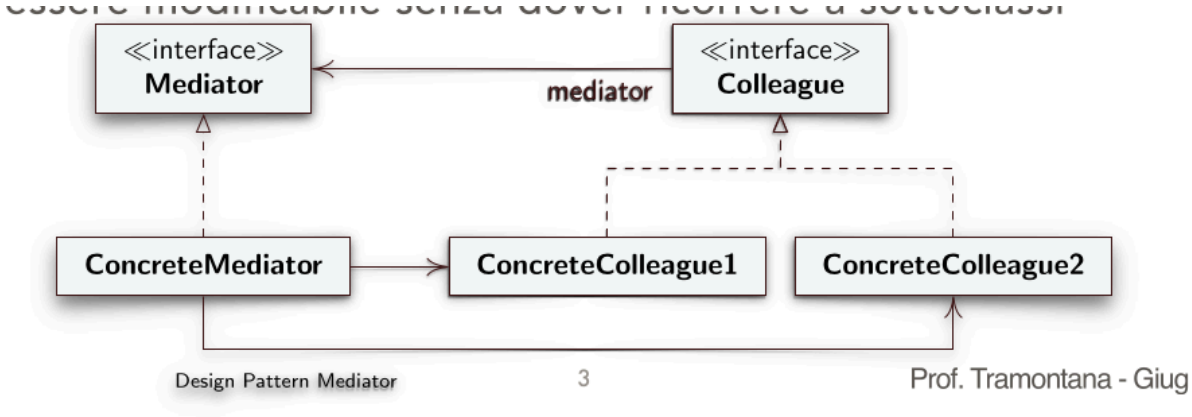




Prof. Tramontana - Giugno 2020

Applicabilità - Quando bisogna usare il Mediator?

1. Un insieme di oggetti **comunicano in modo ben definito ma complesso**. Le interdipendenze che ne risultano sono non strutturate e difficili da comprendere.
2. **Riusare un oggetto è difficile** poiché esso comunica con tanti altri oggetti.
3. Un comportamento che è distribuito fra tante classi dovrebbe essere modificabile anche senza dover ricorrere a sottoclassi. Ovvero **il comportamento collaborativo è sparso fra tutte le classi**, e per cambiarlo dovrei fare delle modifiche in molte classi. Col Mediator, per modificare il comportamento mi basta scrivere un nuovo ConcreteMediator o modificare quello esistente.



Soluzione:

- Isolare le comunicazioni (complesse) tra oggetti dipendenti creando una classe separata per esse.
- **Mediator:** definisce una interfaccia che fa punto di incontro per gli oggetti connessi Colleague.
- **ConcreteMediator:** implementa il comportamento cooperativo e coordina oggetti Colleague. Inoltre, questo mantiene i riferimenti ai suoi Colleague.
- Ogni **Colleague** conosce il Mediator, e comunica con il Mediator quando avrebbe comunicato con un altro Colleague.

- I **ConcreteColleague** mandano richieste (e ricevono richieste) a (e da) un oggetto Mediator. Il Mediator implementa il comportamento cooperativo inoltrando le richieste a opportuni ConcreteColleague.
- Perché Colleague (e quindi i ConcreteColleague) conoscono solo l'interfaccia Mediator generica, e il ConcreteMediator invece conosce i ConcreteColleague specifici? Perché, dal lato suo, il Mediator ha bisogno di chiamare metodi specifici di un ConcreteColleague in particolare, e quindi deve conoscerli. Qui notiamo ovviamente un **accoppiamento**, ma è il costo da pagare per avere il resto del sistema disaccoppiato. Il ConcreteMediator diventa il punto in cui ogni estensione del sistema si paga.
- È importante sapere che, in alcune applicazioni pratiche, **Mediator e ConcreteMediator possono coincidere**. Ciò viene fatto in quanto nella maggior parte dei casi reali non si ha bisogno di più implementazioni intercambiabili di Mediator per lo stesso gruppo di Colleague. Tuttavia, in questo modo perdiamo la possibilità di sostituire il Mediator con un'implementazione diversa senza toccare i Colleague (es. per una **variante del comportamento di coordinamento**).

Conseguenze:

- **Centralizza il controllo**: la maggior parte della complessità che risulta nella gestione delle dipendenze è spostata dagli oggetti cooperanti al Mediator. Questo rende gli oggetti più facili da implementare e mantenere. Questo ovviamente ha anche un aspetto negativo da contemplare: il trade-off è che così il Mediator stesso sarà potenzialmente più complesso di ciascun Colleague preso singolarmente, al punto di poter diventare un monolite difficile da mantenere.
- Le classi Colleague sono **più riusabili** poiché la loro funzionalità fondamentale non è mischiata con il codice che gestisce le dipendenze. In questo modo i Colleague possono essere usati così come sono, senza bisogno di fare sottoclassi per cambiare il modo in cui operano.
- **Il codice del Mediator non è in genere riusabile** poiché la gestione delle dipendenze implementata è solo per una specifica applicazione.
- **Semplifica i protocolli a oggetti**: sostituisce relazioni **many-to-many** tra Colleague con relazioni **one-to-many** tra Mediator e Colleague.

- **Astrae come gli oggetti cooperano**: rende la mediazione un concetto indipendente, incapsulato in un oggetto a sé, permettendo di concentrarsi su come gli oggetti interagiscono separatamente rispetto a cosa fa ciascun oggetto individualmente. Così, leggendo il ConcreteMediator è possibile capire l'intera logica di cooperazione del sistema.

Considerazioni finali:

- **Quantificare "tante dipendenze":** ovviamente non esiste un numero magico che possa indicare quante dipendenze sono tante e quante sono poche. Tuttavia, la soglia indicativa è che più di 5 dipendenze sono sufficienti per poter definire una classe "troppo accoppiata". Questo viola infatti la Law Of Demeter: parla solo con i tuoi amici, non con gli sconosciuti.
- **Dipendenza comportamentale vs Dipendenza strutturale:** il Mediator non elimina il bisogno che gli oggetti dipendano l'uno dall'altro a livello di comportamento. Se Category cambia, Sub-Category deve aggiornarsi, ovviamente questo requisito non scompare. **Quello che il Mediator elimina è la dipendenza strutturale**, ovvero il riferimento diretto.
- Prima del Mediator, la singola classe deve sapere chi avvisare, conoscere le interfacce delle altre classi, e capire cosa passare loro. Dopo l'introduzione del Mediator, la classe diventa più "generica", e tutta la specializzazione che aveva prima legata alla cooperazione con le altre classi si sposta sul Mediator.
- Se nella progettazione c'è una **relazione che forma un loop che torna su sé stessa**, è un segnale che qualcosa nel design è sbagliato. Invece, se riusciamo a disegnare le frecce di un diagramma UML senza farle sovrappore o incrociare, è un indizio che le dipendenze sono gestibili. Il contrario, invece, segnala che le dipendenze sono eccessive e quindi potrebbe avere senso utilizzare il Mediator.
- **Mediator vs Facade:** Facade semplifica un'interfaccia verso un sottosistema con comunicazione **unidirezionale**, e il sottosistema non sa che Facade esiste. Mediator invece centralizza comunicazione **bidirezionale** tra Colleague consapevoli della sua esistenza.
- **Mediator e Observer:** a volte questi D.P. collaborano, in quanto i Colleague potrebbero comunicare col Mediator usando l'Observer (dove i Colleague notificano il Mediator).

Sintesi:

- Il Mediator è un Pattern Comportamentale. Il suo intento è definire un oggetto che incapsula come un intero gruppo di oggetti interagisce fra loro. Promuove il lasco accoppiamento evitando che gli oggetti comunichino direttamente, e permette di modificare le loro interazioni senza dover toccare gli oggetti stessi. Qui non incapsuliamo un singolo comportamento o una singola relazione, ma l'intera rete di interazioni fra più oggetti contemporaneamente.
- Il problema che risolve è quello della **distribuzione di responsabilità** che degenera in troppe connessioni dirette fra oggetti. Nel caso peggiore, un oggetto finisce per conoscere tutti gli altri oggetti, e il sistema si comporta come un monolite, diventando **difficile da modificare**, e diventa difficile pure **riusare singolarmente** le componenti.
- L'idea è quella di **isolare la comunicazione in una classe separata**. Gli oggetti che prima si conoscevano ora smettono di farlo, e conoscono solo l'interfaccia Mediator. Ovviamente il Mediator **non elimina la dipendenza comportamentale**, ma solo quella strutturale.
- Raramente serve avere più implementazioni intercambiabili di Mediator, e quindi spesso Mediator e ConcreteMediator finiscono per coincidere.
- I Colleague così diventano più semplici e riusabili, ma di contro il Mediator diventa complesso e tipicamente non è riusabile, visto che la logica di coordinamento che implementa riguarda una specifica applicazione. Il Pattern sostituisce relazioni **many-to-many** fra Colleague in relazioni **one-to-many** fra Mediator e Colleague. Questo Pattern soddisfa il principio **Law Of Demeter**.

Progettare per Preservare

Principio generale: Programmare per le interfacce, non per le implementazioni.

- Creare un oggetto specificandone la classe esplicitamente **orienta a una particolare implementazione invece che a una interfaccia**, e questo può complicare i cambiamenti futuri.
  È meglio creare oggetti indirettamente.
  Pattern utili: Factory Method, Prototype.

- Le **dipendenze da operazioni specifiche vincolano a un modo di soddisfare una richiesta**, evitando che le richieste siano indicate nel codice permette di modificare a compile-time e a run-time il modo in cui le richieste saranno soddisfatte. Il problema è che il codice client chiama direttamente un'operazione precisa per soddisfare la richiesta (es. con una catena di if/else).
  Pattern utili: Chain Of Responsibility, Command.
- **Dipendenze da piattaforme hardware e software**, ovvero API esterne specifiche di una piattaforma, rendono più difficile fare il porting del sistema software verso altre piattaforme. È persino difficile tenerlo aggiornato per la stessa piattaforma. Bisogna progettare il sistema limitando le dipendenze dalla piattaforma.
  Pattern utili: Factory, Bridge. Possiamo separare l'astrazione dall'implementazione.
- **Dipendenze da rappresentazioni o implementazioni** (come un oggetto è rappresentato, conservato o implementato) rendono le classi client soggette a cambiamenti quando l'oggetto cambia. Bisogna nascondere le informazioni alle classi client, e in questo modo evitare cambiamenti a cascata.
  Pattern utili: Factory, Bridge.
- **Dipendenze di oggetti da algoritmi** che sono spesso estesi, ottimizzati e rimpiazzazti durante lo sviluppo e il riuso, provocando cambiamenti agli oggetti nel caso di cambiamento degli algoritmi. Si devono isolare gli algoritmi soggetti a cambiamenti. In questo modo, ogni volta che l'algoritmo cambia bisogna modificare l'oggetto, per questo conviene isolare l'algoritmo in una gerarchia di classe a parte, intercambiabile.
  Pattern utili: State/Strategy, Template Method.
- **Stretto accoppiamento fra varie classi** porta a sistemi monolitici. **Classi strettamente accoppiate** sono difficili da modificare, e riusare singolarmente, senza comprendere e cambiare le altre classi.
  Il lasco accoppiamento incrementa la probabilità che una classe sia riusata da sola, e che il sistema sia compreso, modificato ed esteso più facilmente.
  Relazionare bene le classi è difficile, le chiamate dirette creano dipendenze che si accumulano e complicano il sistema.

Pattern utili: Abstract Factory, Bridge, Chains of Responsibility, Command, Facade, Mediator, Observer.

- **L'estensione di funzionalità tramite sottoclassi** è **riuso a scatola bianca**, e richiede profonda comprensione della superclasse e l'override di ciascuna operazione può richiedere l'override di un'altra. Può condurre a un grande numero di classi (per estensioni).
  È meglio **limitare il numero di livelli in una gerarchia** di classi e usare la composizione anziché l'ereditarietà: **black box reuse**.
  Alcuni design pattern producono una progettazione in cui si possono personalizzare le funzionalità definendo solo una sottoclasse e componendo le sue istanze con le altre.
  Pattern utili: Bridge, Chain Of Responsibility, Composite, Decorator, Observer, Strategy.
- Quando è **impossibile modificare classi**, per es. perché il codice sorgente non è disponibile, o quando una modifica richiede cambiamenti a tante altre classi.
  Pattern utili: Adapter, Decorator.

State vs Strategy

Le relazioni strutturali sono le stesse. L'unica cosa che cambia è l'**intento**. Nello State il sistema è pensato come una macchina a stati che transita da uno stato all'altro nel tempo, cambiando il comportamento di conseguenza. Nello Strategy, invece, l'obiettivo è scegliere fra algoritmi diversi per fare la stessa cosa.

Runtime: Lo strategy cambia regolarmente a runtime. Per esempio, potremmo voler scegliere un algoritmo di ordinamento in base alla dimensione dei dati, decisione che può benissimo essere presa più volte durante l'esecuzione.

Differenza reale: chi decide il cambiamento e perché. Nello State, di solito sono gli stessi oggetti ConcreteState a conoscersi a vicenda e a decidere quando passare allo stato successivo. Nello Strategy, è il Client o il Context dall'esterno a scegliere quale algoritmo usare in base a una configurazione o a una condizione. Le ConcreteStrategy normalmente non si conoscono fra loro e non decidono autonomamente di "passare il turno" a un'altra strategy. Il criterio giusto è "**chi guida la transizione e con quale intento**". Anche qui in realtà bisogna fare attenzione, perché esiste una variante dello State in cui è il Context a comunicare ai ConcreteState il cambiamento. I ConcreteStrategy, invece, non possono

decidere il cambiamento fra di loro, ma deve essere per forza il Context a comunicare loro la transizione. Quindi **la vera differenza è l'intento**.